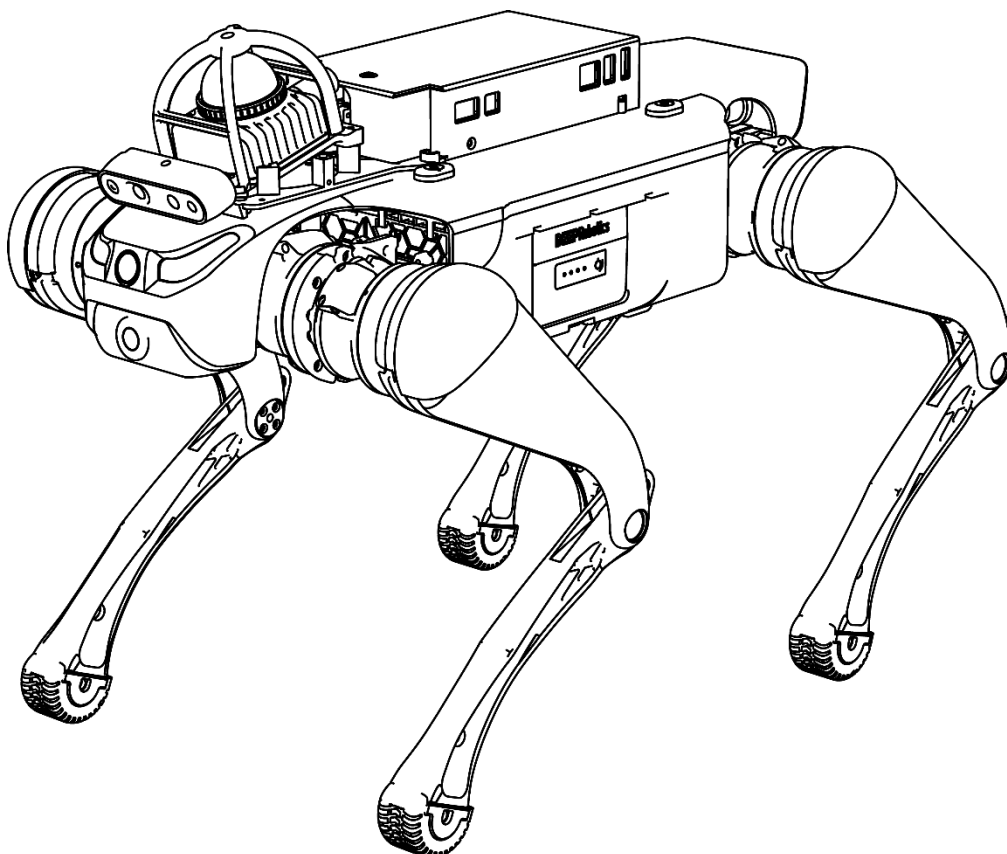


绝影 Lite3 感知开发手册(beta)

V2.2.3 2025.05.06



目录

1 感知系统	6
2 准备工作	7
2.1 远程登录	7
2.1.1 连接	7
2.1.2 故障排查	8
2.2 通过 HDMI 连接感知主机	8
2.2.1 设置图形化界面自启动	8
2.2.2 恢复 tty3 自启动	10
3 ROS 版本查看与切换	11
4 深度相机	12
4.1 相机驱动	12
4.2 相机测试	13
4.3 librealsense 二次开发	13
4.4 Realsense-ROS1 二次开发	14
4.5 Realsense-ROS2 二次开发	14
5 运动通信功能包	15
5.1 功能介绍	15
5.2 使用方法	16
5.2.1 ROS1	16
5.2.2 ROS2	18
5.3 程序结构说明	19

5.3.1 ROS1	19
5.3.2 ROS2	20
6 识别跟随功能包	23
6.1 功能介绍	23
6.2 使用方法	24
6.3 二次开发说明	24
7 基于激光雷达的建图定位导航避障(ROS1)	26
7.1 建图（适用于 v3.1.04 及以后）	26
7.1.1 功能介绍	26
7.1.2 程序结构	26
7.1.3 使用方法	27
7.2 建图（适用于 v3.1.04 以前）	32
7.2.1 功能介绍	32
7.2.2 程序结构	32
7.2.3 使用方法	33
7.3 定位导航	37
7.3.1 单目标点定位导航使用方法	37
7.3.2 多目标点定位导航使用方法	39
7.4 二次开发说明	40
7.4.1 雷达驱动二次开发	40
7.4.2 SLAM 建图二次开发	40
7.4.3 定位导航二次开发	41

8 基于激光雷达的建图定位导航避障(ROS2).....	42
8.1 建图	42
8.1.1 功能介绍	42
8.1.2 程序结构	42
8.1.3 使用方法	42
8.2 定位导航.....	47
8.2.1 单目标点定位导航使用方法.....	48
8.2.2 多目标点定位导航使用方法.....	49
8.3 二次开发说明	51
8.3.1 雷达驱动二次开发	51
8.3.2 SLAM 建图二次开发.....	51
8.3.3 定位导航二次开发	51

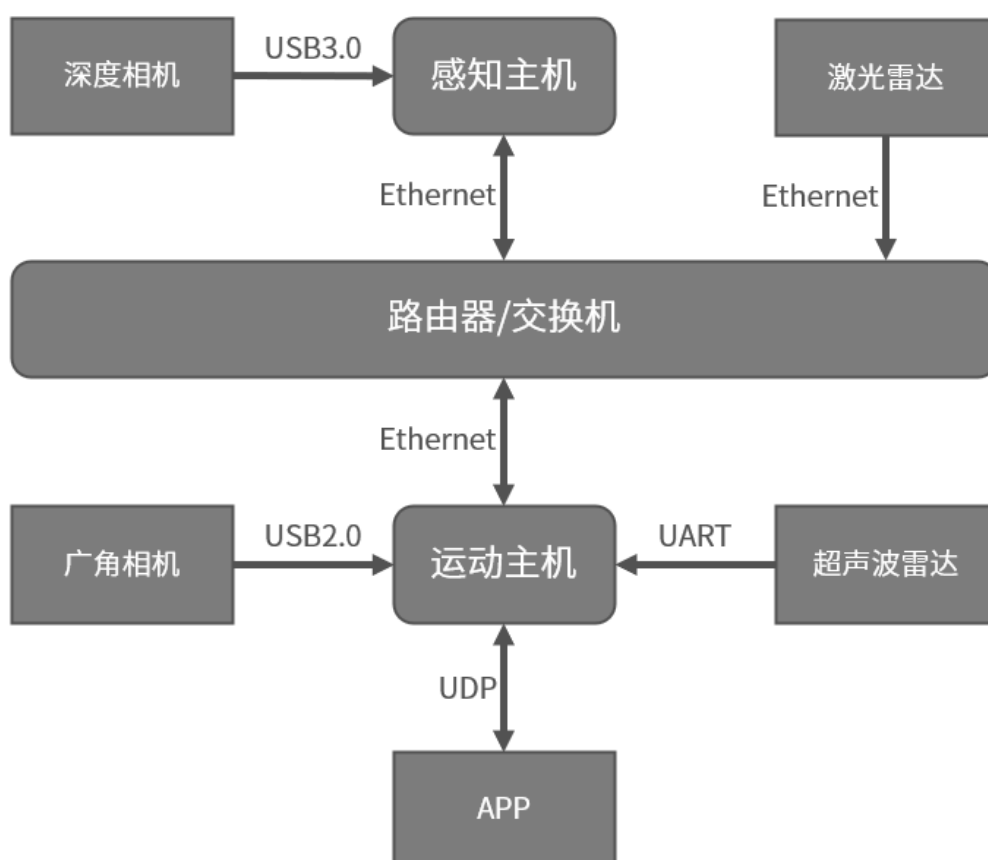
文档说明

本手册适用于需要通过绝影 Lite3 平台对感知算法进行探索、开发和验证，并且具备一定专业知识的用户。本手册适用于 Ubuntu20 的版本。

手册版本	更新说明	发布日期
V1.0.1	初版发布	2023/5/25
V1.0.2	修改和补充	2023/6/16
V1.1.1	升级 ubuntu20	2024/1/3
V1.1.2	升级至 Yolov8	2024/3/8
V1.1.3	完善建图导航跟随步骤细节	2024/4/11
V2.0.0	修正部分细节，适用 U20	2024/5/10
V2.0.1	修正部分细节	2024/5/15
V2.1.0	新增新版建图操作	2024/6/29
V2.1.1	新增虚拟桌面开关操作	2024/7/10
V2.2.1	新增 ROS2	2024/9/3
V2.2.2	新增建图前查看 ROS 版本注意事项； 补充远程桌面使用说明； 补充修改地图名字和路径时的操作	2024/12/31
V2.2.3	更新激光版配图； 更新不同雷达的驱动启动方法； 更新二次开发的方法说明	2025/5/6

1 感知系统

绝影 Lite3 专业版和激光版配备了感知主机 NVIDIA Jetson Xavier NX，并配有感知算法案例，以便于用户进行二次开发。各传感器之间的连接情况如图所示，运动主机和感知主机通过 UDP 进行数据传输。



2 准备工作

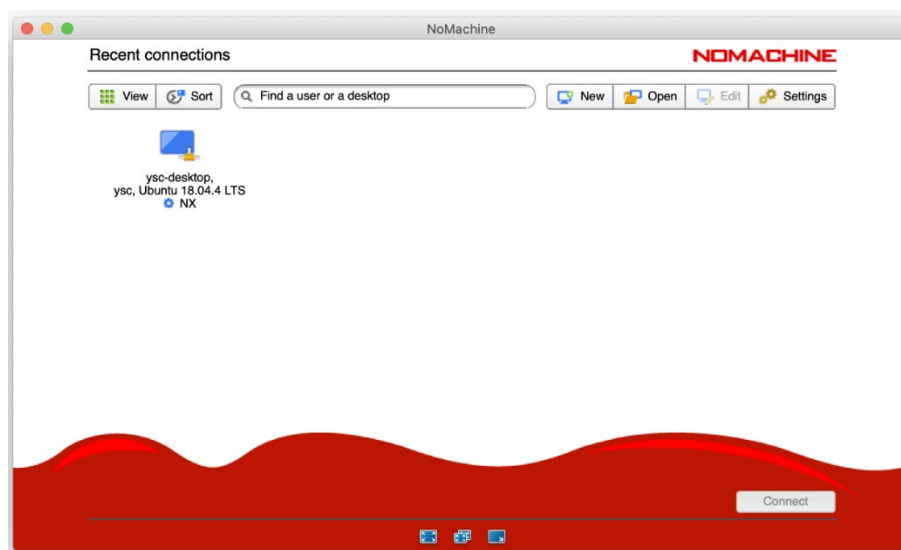
2.1 远程登录

【注意】如果在使用远程桌面前，已参考 2.2 节对 HDMI 连接进行了配置，需参照 2.2.2 节说明对配置进行恢复并重启机器狗，否则无法使用远程桌面。

2.1.1 连接

用户可通过 NoMachine 软件远程登录绝影 Lite3 感知主机桌面。方法如下：

1. 首先把开发主机连接到机器狗 WiFi；
2. 然后在开发主机上打开 NoMachine 软件，点击 "New" / "Add" 新建连接；
 - a) “Protocol” 选项选择 “NX”
 - b) “Host” 栏目填写 “192.168.1.103”
 - c) “Authentication” 选项选择 “Use password authentication”
 - d) “Proxy” 选项不勾选 “Use custom proxy configuration for this session”
 - e) 其余选项保持默认
3. 完成后页面将出现一个新的远程图标，如下图所示；



4. 点击图标，输入用户名 `yssc` 和密码 `'`（英文单引号），即可进行远程连接。

【注意】连接后进行操作时，如果遇到锁屏界面，或者终端命令要求输入密码时，密码为 `'`（英文单引号）。

2.1.2 故障排查

1. 若输入密码 `'` 时显示密码错误，请尝试敲击一次 shift 按键切换英文输入法后，再输入 `'`。
2. 若连接后远程桌面白屏，请点击 NoMachine 右上角的“Settings”，弹出设置页面后，选择左下角“Server Preferences”，再选择右上方“Updates”，点击“Check now”进行更新。更新完成后再次尝试远程连接。若输入密码后显示“session negotiation failed”，则需要在本地图 SSH 登录进行修复：

```
1 ssh yssc@192.168.1.103      #密码是单引号
2 sudo su
3 cd /usr/NX/var/db/limits/
4 ls                          #列出 limits 目录下的文件
5 rm xxxxxx xxxxxx xxxxxx    #rm 为删除指令，xxxxxx 为目录下的各个文件名
```

若以上操作之后还是显示“session negotiation failed”，则再次进行以上操作，并使用

`sudo reboot` 重启感知主机。

2.2 通过 HDMI 连接感知主机

绝影 Lite3 专业版/激光版支持通过背部 HDMI 口连接到感知主机桌面。感知主机开机时默认进入 tty3 终端，如需开机后自动进入图形化界面，需要进行相关设置。

2.2.1 设置图形化界面自启动

1. 首先使用 HDMI 线连接感知主机与显示器并将机器狗开机后，系统会进入开机界面，当系统出现 “[OK] Started Session 1 of user yssc.” 后即为开机成功。

```
[ 25.916205] Bridge firewalling registered
[ OK ] Started User Manager for UID 1000.
[ OK ] Started Session 1 of user yssc.
```

【注意】若开机后未出现 NVIDIA 标识或 “[OK] Started Session 1 of user ysc.”，而是出现了 “[Failed]” 开头的字样，可能为感知模块硬件出现故障，请联系售后人员进行处理。

2. 开机成功后，使用 USB 接口连接键盘，按 Ctrl+Alt+F3 按键进入 tty3 终端。

```
Ubuntu 20.04.6 LTS lite tty3
```

```
lite login:
```

3. 随后输入用户名 `ysc` 和密码 `'`（英文单引号），进行登录。

```
Ubuntu 20.04.6 LTS lite tty3
```

```
lite login: ysc
```

```
Password:
```

```
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.10.120-tegra aarch64)
```

```
* Documentation: https://help.ubuntu.com
```

```
* Management:   https://landscape.canonical.com
```

```
* Support:       https://ubuntu.com/advantage
```

```
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.
```

```
To restore this content, you can run the 'unminimize' command.
```

```
Expanded Security Maintenance for Applications is not enabled.
```

```
222 updates can be applied immediately.
```

```
107 of these updates are standard security updates.
```

```
To see these additional updates run: apt list --upgradable
```

```
68 additional security updates can be applied with ESM Apps.
```

```
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
```

4. 登录成功后（如上图所示），进入 `/usr/share/X11/xorg.conf.d` 目录并将 `xorg.conf` 文件移至上一目录，具体操作如下：

```
1 | cd /usr/share/X11/xorg.conf.d/
```

```
2 | sudo mv xorg.conf .. #mv 为移动指令，将 xorg.conf 文件移至上一目录
```

并在 “[sudo] password for ysc:” 后输入密码 `'`（英文单引号）。

```
ysc@lite:~$ cd /usr/share/X11/xorg.conf.d/
```

```
ysc@lite:/usr/share/X11/xorg.conf.d$ sudo mv xorg.conf ..
```

```
[sudo] password for ysc:
```

```
ysc@lite:/usr/share/X11/xorg.conf.d$ _
```

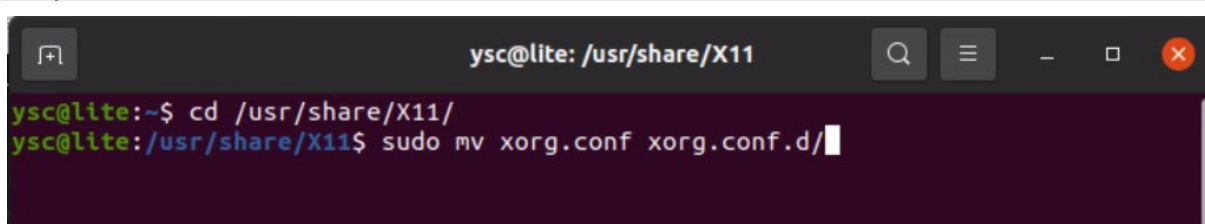
5. 重启机器狗后，感知主机会自动进入图形化界面，如果未出现图形化界面，可能是因为系统未清除之前配置的缓存，请再次重启机器狗。

2.2.2 恢复 tty3 自启动

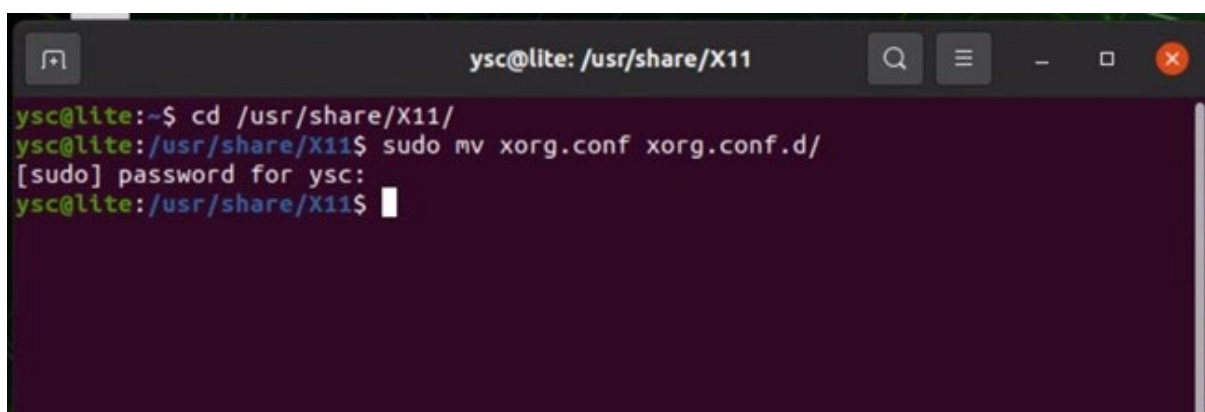
将 xorg.conf 恢复至原路径并重启机器狗，即可恢复 tty3 自启动。方法如下：

1. 开机成功进入图形化界面后，按 Alt+Ctrl+T 按键打开终端窗口，并输入以下内容：

```
1 | cd /usr/share/X11/  
2 | sudo mv xorg.conf xorg.conf.d/ #mv 为移动指令，将 xorg.conf 文件移至 xorg.conf.d/下
```



2. 输入密码'（英文单引号），进行文件移动。若移动失败，可能为 xorg.conf 文件不在“/usr/share/X11/”下，寻找 xorg.conf 文件的实际所在位置，并将其移动至目录“/usr/share/X11/xorg.conf.d/”下。



3. 重启机器狗，即可恢复 tty3 自启动，开机后，感知主机将默认进入 tty3 终端。

3 ROS 版本查看与切换

绝影 Lite3 从感知主机镜像 V3.2.1 版本开始同时支持 ROS1 和 ROS2。

【注意】用户可查看感知主机中的~/Desktop/version_log.txt 获取镜像版本信息。

1. 在终端执行以下命令，以查看当前使用的 ROS 版本：

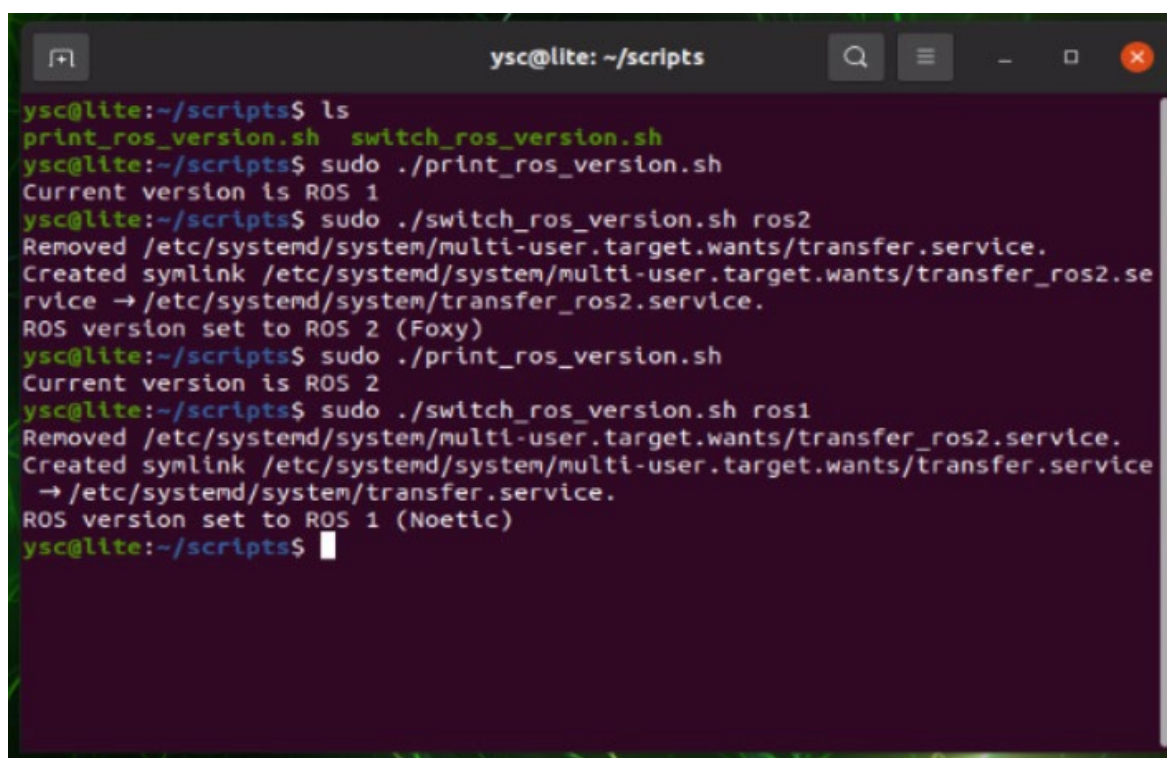
```
1 | cd /home/ysc/scripts
2 | sudo ./print_ros_version.sh
```

2. 在终端执行以下命令，以切换至 ROS1 版本：

```
1 | cd /home/ysc/scripts
2 | sudo ./switch_ros_version.sh ros1
```

3. 在终端执行以下命令，以切换至 ROS2 版本：

```
1 | cd /home/ysc/scripts
2 | sudo ./switch_ros_version.sh ros2
```

A terminal window titled 'ysc@lite: ~/scripts' showing the execution of ROS version switching scripts. The user first lists files, then runs 'print_ros_version.sh' which reports 'Current version is ROS 1'. Next, they run 'switch_ros_version.sh ros2', which shows systemd service changes and sets the version to 'ROS 2 (Foxy)'. Finally, they run 'print_ros_version.sh' again, which reports 'Current version is ROS 2'. Then they run 'switch_ros_version.sh ros1', showing another set of systemd service changes and setting the version to 'ROS 1 (Noetic)'. The terminal ends with the prompt 'ysc@lite:~/scripts\$'.

4. 切换成功后，请重启机器狗。

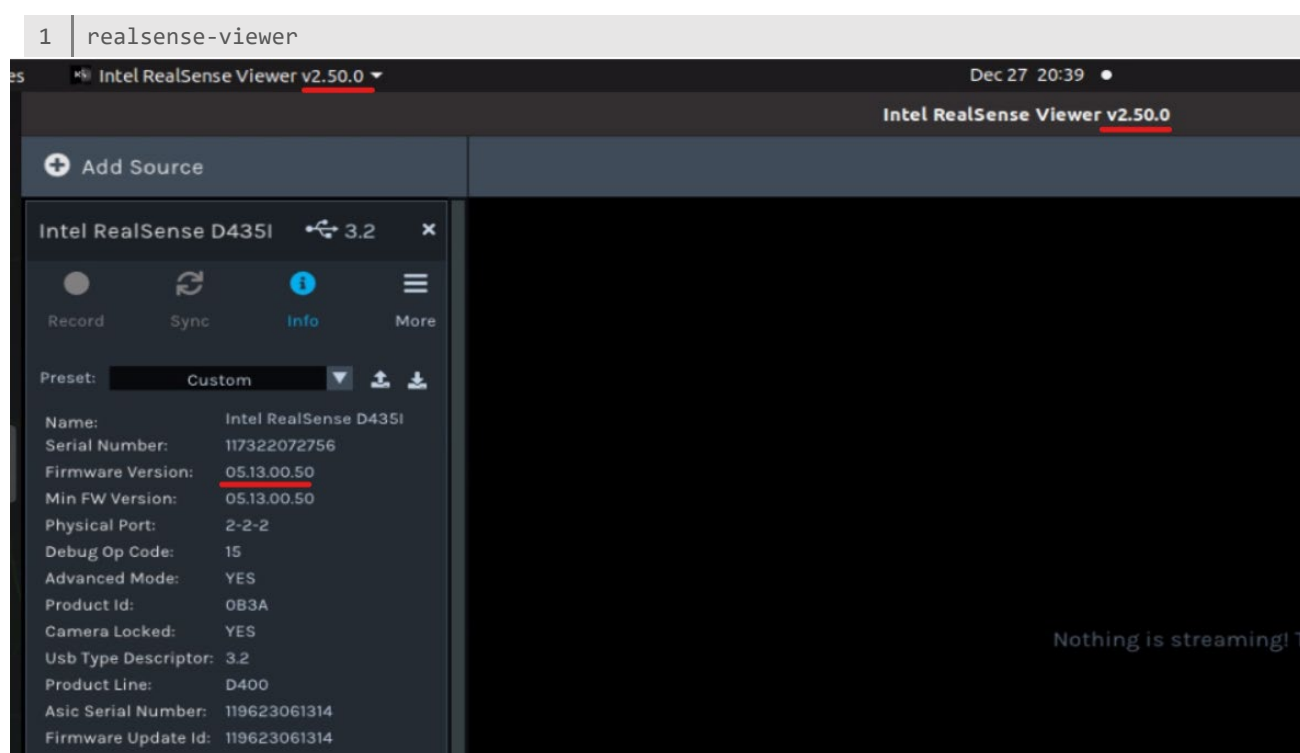
【注意】请勿同时运行 ROS1 和 ROS2 版本程序，以免造成不必要的资源占用和报错。

4 深度相机

绝影 Lite3 专业版和激光版配备了深度相机 Intel RealSense D435i。

4.1 相机驱动

绝影 Lite3 的感知主机上已经安装了相应的深度相机驱动程序 Intel RealSense SDK，用户可在终端输入以下命令行打开 Intel 官方提供的可视化程序：



点击 Info 按钮，可以看到更详细的参数信息，例如相机的固件版本、序列号等。

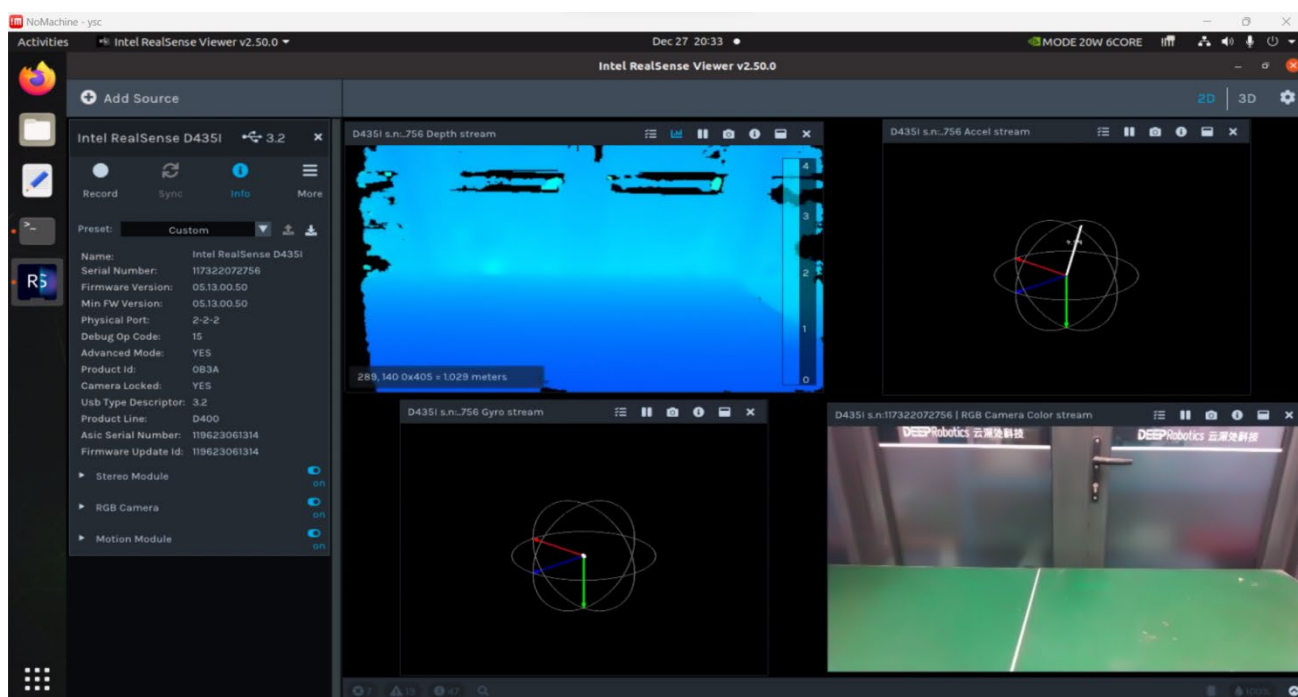
【注意】realsense-ros 包依赖 v2.50.0 版本的 librealsense 库，其对应的深度相机固件版本为 05.13.00.50。librealsense 库的版本在 realsense-viewer 软件标题处显示，固件版本在 Info 标签页的 Firmware Version 处显示。

展开 Stereo Module 或者 RGB Camera，可以配置相机的分辨率、帧数等参数并进行实时效果查看。

4.2 相机测试

在使用相机驱动进行开发前，先检查深度相机是否正常连接：

1. 确认左侧栏目中已经显示 Intel RealSense D435i；
2. 点击 Stereo Module 和 RGB Camera 的 on/off 开关，若成功打开深度图和彩色图，说明深度相机正常连接。



4.3 librealsense 二次开发

librealsense 相关库基于 CUDA 进行编译，已安装至/usr/local/lib 和/usr/local/include 下，若要进行相关二次开发可自行 include 对应头文件并链接对应库进行编译。

```
ysc@lite:/usr/local/include$ ls
GLFW          librealsense2-gl  livox_lidar_cfg.h
librealsense2 livox_lidar_api.h livox_lidar_def.h
```

```
ysc@lite:/usr/local/lib$ ls
cmake          librealsense2-gl.so.2.50      ocaml
libfw.a        librealsense2-gl.so.2.50.0    pkgconf
libglfw3.a     librealsense2.so             python2.7
liblivox_lidar_sdk_shared.so librealsense2.so.2.50        python3.8
liblivox_lidar_sdk_static.a librealsense2.so.2.50.0      python3.9
librealsense2-gl.so librealsense-file.a
```

realsense-ros 包位于/home/ysc/lite_cog/drivers/realsense_ws 目录下，相关功能可通过系统服务开启，对应命令行为：`sudo systemctl start realsense`，该命令将使用/home/ysc/lite_cog/drivers/realsense_ws/src/realsense2_camera/launch 文件夹下的 dr_camera.launch 文件进行 realsense 相机的启动，若需要修改相机的启动参数，可自行修改该 launch 文件。

4.4 Realsense-ROS1 二次开发

ROS1 版本的 realsense-ros 包位于/home/ysc/lite_cog/drivers/realsense_ws 目录下，相关功能可通过系统服务开启关闭和查看状态，对应命令行为：

```
1 | sudo systemctl start realsense
2 | sudo systemctl stop realsense
3 | sudo systemctl status realsense
```

命令将使用~/lite_cog/drivers/realsense_ws/src/realsense2_camera/launch 文件夹下的 dr_camera.launch 文件进行 realsense 相机的启动，若需要修改相机的启动参数，可自行修改该 launch 文件。

4.5 Realsense-ROS2 二次开发

ROS2 版本的 realsense-ros 包位于/home/ysc/lite_cog_ros2/driver/realsense_ws 目录下，相关功能可通过系统服务开启关闭和查看状态，对应命令行为：

```
1 | sudo systemctl start realsense_ros2
2 | sudo systemctl stop realsense_ros2
3 | sudo systemctl status realsense_ros2
```

命令将使用~/lite_cog_ros2/drivers/realsense_ws/src/realsense2_camera/launch 文件夹下的 dr_camera_launch.py 文件进行 realsense 相机的启动，若需要修改相机的启动参数，可自行修改该 launch.py 文件。

5 运动通信功能包

5.1 功能介绍

本案例实现了 ROS 与 UDP 消息的功能转换。

绝影 Lite3 的运动主机与感知主机之间、感知主机与手柄 APP 之间的数据传输均采用 UDP 协议，通过本案例提供的 *message_transformer_cpp* 软件包，可实现：

1. 将运动主机上报的 UDP 消息转换为 ROS 话题进行发布，将感知主机下发的运动控制指令使用 UDP 发送给运动主机；
2. 接收 APP 发送的控制指令，以开启和关闭感知主机上的 AI 功能。

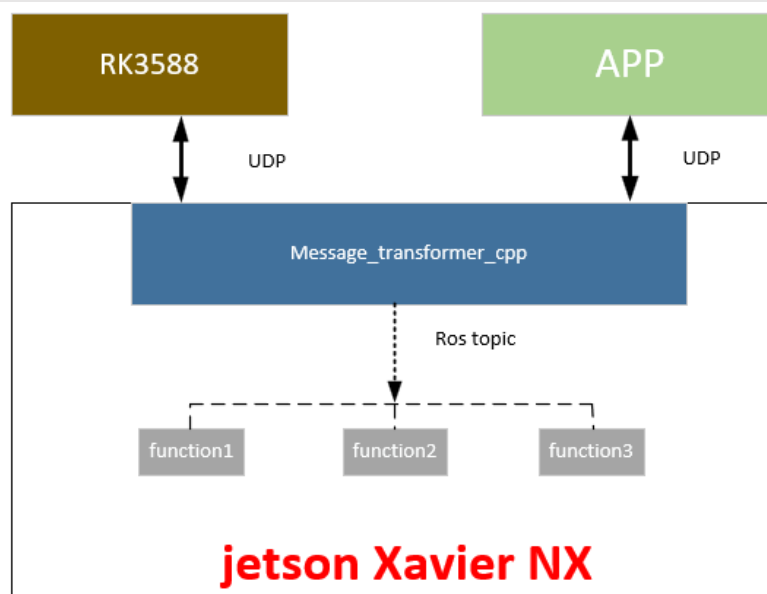
其提供的 ROS 通信接口：

发布话题： 运动主机向感知主机传输数据

腿部里程计：	/leg_odom	(nav_msgs::Odometry)
IMU 数据：	/imu/data	(sensor_msgs::Imu)
关节数据：	/joint_states	(sensor_msgs::JointState)

订阅话题： 感知主机向运动主机传输数据

速度指令：	/cmd_vel	(geometry_msgs::Twist)
-------	----------	------------------------



- a) 用户可在基于 ROS 编译的 C++和 Python 程序中发布该话题(ROS 基础的学习请参考 <http://wiki.ros.org/ROS/Tutorials>), 也可以打开一个终端, 输入命令发布进行调试, 请先输入:

```
1 | rostopic pub /cmd_vel geometry_msgs/Twist
```

- b) 输入完先不运行, 在语句后面加一个空格, 再按 Tab 键, 就会自动补充当前消息类型的内容, 具体如下:

```
1 | rostopic pub /cmd_vel geometry_msgs/Twist "linear:
2 | x: 0.0
3 | y: 0.0
4 | z: 0.0
5 | angular:
6 | x: 0.0
7 | y: 0.0
8 | z: 0.0
9 | "
```

- c) 使用键盘上的向左/向右方向键移动光标修改速度值, 然后在 `geometry_msgs/Twist` 之后加入 `-r 10` 规定发布频率(即每秒发布 10 次), 输入完毕后终端内容如下所示:

```
1 | rostopic pub /cmd_vel geometry_msgs/Twist -r 10 "linear:
2 | x: 0.2
3 | y: 0.1
4 | z: 0.0
5 | angular:
6 | x: 0.0
7 | y: 0.0
8 | z: 0.3
9 | "
```

- d) 运行命令, 即可发布话题。
- e) 传输程序会订阅该话题, 并将其转为 UDP 指令消息发给运动主机。
- f) 传输程序正常开启后, 使机器狗起立, 并启动 APP 设置页中的自主模式, 机器狗即可按照如上速度行动。

【注意】请在空旷处调试以防对人或物品造成损伤, 遇到紧急情况, 及时按下软急停, 或关闭自主模式。

- a) 用户可在基于 ROS2 编译的 C++和 Python 程序中发布该话题(ROS 基础的学习请参考 <https://docs.ros.org/en/foxy/Tutorials.html>), 也可以打开一个终端, 输入命令

发布进行调试, 请输入:

```
1 | ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

- b) 运行命令, 即可发布话题。
- c) 传输程序会订阅该话题, 并将其转为 UDP 指令消息发给运动主机。
- d) 传输程正常开启后, 使机器狗起立, 并启动 APP 设置页中的自主模式, 机器狗即可按照如上速度行动。

【注意】请在空旷处调试以防对人或物品造成损伤, 遇到紧急情况, 及时按下软急停, 或关闭自主模式。

5.3 程序结构说明

5.3.1 ROS1

```
/home/ysc/lite_cog/transfer/src
├─ CMakeLists.txt
├─ message_transformer
│   ├── CMakeLists.txt
│   ├── include
│   │   ├── protocol.h
│   │   └─ sensor_logger.h
│   ├── launch
│   │   └─ message_transformer.launch
│   └─ msg
│       ├── SimpleCMD.msg
│       └─ ComplexCMD.msg
├─ package.xml
└─ src
    ├── nx2app.cpp
    ├── qnx2ros.cpp
    ├── ros2qnx.cpp
    └─ sensor_checker.cpp
```

1. *nx2app.cpp* 主要用于感知主机与手柄 APP 之间进行 UDP 通信，APP 下发指令到感知主机，该程序根据收到的命令执行相应的操作。手柄与感知主机之间通信所用的指令结构体如下：

```

1 class SimpleCMD{
2 public:
3     int32_t cmd_code;
4     int32_t cmd_value;
5     int32_t type;
6 };

```

2. *qnx2ros.cpp* 用于接收运动主机上报的数据，并将其转化为 ROS 话题，供其他功能包调用，发布的话题如下：

```

1 leg_odom_pub_ = nh.advertise<geometry_msgs::PoseWithCovarianceStamped>("leg_odom", 1);
2 leg_odom_pub2_ = nh.advertise<nav_msgs::Odometry>("leg_odom2", 1);
3 joint_state_pub_ = nh.advertise<sensor_msgs::JointState>("joint_states", 1);
4 imu_pub_ = nh.advertise<sensor_msgs::Imu>("/imu/data", 1);
5 handle_pub_ = nh.advertise<geometry_msgs::Twist>("/handle_state", 1);
6 ultrasound_pub_ = nh.advertise<std_msgs::Float64>("/us_publisher/ultrasound_distance",
1);

```

3. *ros2qnx.cpp* 订阅其他功能包节点发布的话题，转化为 UDP 数据报下发给运动主机，订阅的话题如下：

```

1 ros::Subscriber vel_sub = nh.subscribe("cmd_vel", 1, &ROS2QNX::CmdVelCallback, &ros2qnx);
2 ros::Subscriber vel_sub2 = nh.subscribe("cmd_vel_corrected", 1, &ROS2QNX::CmdVelCallback,
3 &ros2qnx);
4 ros::Subscriber simplecmd_sub = nh.subscribe("simple_cmd", 1,
5 &ROS2QNX::SimpleCMDCallback, &ros2qnx);
6 ros::Subscriber complexcmd_sub = nh.subscribe("complex_cmd", 1,
7 &ROS2QNX::ComplexCMDCallback, &ros2qnx);

```

5.3.2 ROS2

```
ysc@lite:~/lite_cog_ros2/transfer/src$ tree .
```

```

|— transfer
|   |— CMakeLists.txt
|   |— include
|   |   |— protocol.hpp
|   |— launch
|   |   |— transfer_launch.py

```

```

|   ├── package.xml
|   └── src
|       ├── Jetson2App.cpp
|       ├── Jetson2Motion.cpp
|       ├── SensorChecker.cpp
|       └── SensorsLogger.hpp
└── transfer_interfaces
    ├── CMakeLists.txt
    ├── msg
    │   ├── MotionComplexCMD.msg
    │   └── MotionSimpleCMD.msg
    └── package.xml

```

1. **Jetson2App.cpp** 主要用于感知主机与手柄 APP 之间进行 UDP 通信，APP 下发指令到感知主机，该程序根据收到的命令执行相应的操作。手柄与感知主机之间通信所用的指令结构体如下：

```

1  class SimpleCMD{
2  public:
3      int32_t cmd_code;
4      int32_t cmd_value;
5      int32_t type;
6  };

```

2. **Jetson2Motion.cpp** 中包含 **MotionReceiver** 和 **MotionSender** 两个类。其中 **MotionReceiver** 用于接收运动主机上报的数据，并将其转化为 ROS 话题，供其他功能包调用，发布的话题如下：

```

1  leg_odom_pub_ =
    create_publisher<geometry_msgs::msg::PoseWithCovarianceStamped>("leg_odom", 10);
2  leg_odom_pub2_ = create_publisher<nav_msgs::msg::Odometry>("leg_odom2", 10);
3  joint_state_pub_ = create_publisher<sensor_msgs::msg::JointState>("joint_states", 10);
4  imu_pub_ = create_publisher<sensor_msgs::msg::Imu>("/imu/data", 10);
5  handle_pub_ = create_publisher<geometry_msgs::msg::Twist>("/handle_state", 10);
6  ultrasound_pub_ =
    create_publisher<std_msgs::msg::Float64>("/us_publisher/ultrasound_distance", 10);

```

MotionSender 订阅其他功能包节点发布的话题，转化为 UDP 数据报下发给运动主机，订阅的话题如下：

```

1  cmd_vel_sub_ = create_subscription<geometry_msgs::msg::Twist>(
2      "cmd_vel", 10,

```

```
3     std::bind(&MotionSender::CmdVelCallback, this, std::placeholders::_1));
4 cmd_vel_corrected_sub_ = create_subscription<geometry_msgs::msg::Twist>(
5     "cmd_vel_corrected", 10,
6     std::bind(&MotionSender::CmdVelCallback, this, std::placeholders::_1));
7 simplecmd_sub = create_subscription<transfer_interfaces::msg::MotionSimpleCMD>(
8     "simple_cmd", 10,
9     std::bind(&MotionSender::SimpleCMDCallback, this, std::placeholders::_1));
10 complexcmd_sub = create_subscription<transfer_interfaces::msg::MotionComplexCMD>(
11     "complex_cmd", 10,
12     std::bind(&MotionSender::ComplexCMDCallback, this, std::placeholders::_1));
```

6 识别跟随功能包

6.1 功能介绍

本案例基于 DeepStream、Yolov8 和 TensorRT 进行相机视频流获取、人体识别和目标追踪，计算目标位置并下发给控制模块，实现了机器狗的人体自动识别跟随功能。基于 DeepStream 硬解码获取 h264 编码的 720p 分辨率 rtsp 视频流，使用 TensorRT 加速 Yolov8 人体识别模型以识别开阔的场景中的人，最终识别帧率能达到 20fps 左右，追踪帧率能达到 10fps 左右。

本案例提供的识别跟随功能分为**识别**和**跟踪**两部分：

1. **识别算法**通过深度学习的神经网络进行视觉识别，找到画面中的人体位置。当在画面中出现多个人体时，首先识别出画面中的所有人体，然后对每一帧视频中识别出的人体基于深度学习提取特征，并进行逐一比对，以确定前后帧中同一个人的轨迹。
2. **跟踪算法**允许用户自主选择画面中想要跟随的目标，并能实时锁定目标，持续跟踪。根据视觉识别算法得到的人和机器狗之间大致的角度（方位）、距离等信息，机器狗能够实时判断人狗方位，作出相应的响应（平移、转向），还能根据被追踪者与狗之间的距离实时调整速度：
 - a) Too close：当目标过近时，机器狗将停止前进，防止撞人。
 - b) Close：当目标较近时，机器狗将实时动态调整前进速度，以接近目标。
 - c) Far：当目标较远时，机器狗将以最大速度前进，进行跟随。

本案例中使用的 yolov8 和 sdk_hub 代码来源 <https://github.com/ultralytics/ultralytics> 和 <https://github.com/ultralytics/hub-sdk>，互联网 Yolov8 的学习资料众多，用户可自行搜索学习。

6.2 使用方法

【注意】功能开启时需要进行 rtsp 视频流硬件解码和深度学习推理环境的初始化，耗时大概 40s 左右，请耐心等待，如果长时间未能开启，请使用手柄连接机器狗检查视频流是否正常工作。

1. 打开一个终端输入以下命令，以启动跟随程序：

```
1 | cd /home/ysc/lite_cog/track/src    #ROS1
2 | cd /home/ysc/lite_cog_ros2/track/src  #ROS2
3 | python3 run_tracker.py
```

2. 使用 APP 使机器狗起立并启动自主模式。
3. 当画面中出现人时，系统会为识别到的所有人物分配数字编号，并显示在画面上，用键盘输入需要跟随的人物的编号，按回车锁定对象。

【注意】请始终保持演示窗口为前置状态，确保按键输入是在演示窗口界面进行。

4. 随后机器狗将对目标进行追踪、识别。
5. 跟随期间或目标丢失显示“Miss Object”时可以按回车进行重置。
6. 在终端中按下 CTRL+C 键结束程序。

6.3 二次开发说明

本案例提供了两个不同版本的 *people_tracking* 功能包，分别放置在 `~/lite_cog/track` 和 `~/lite_cog_ros2/track` 工作空间中。二者的唯一区别在于 `/src/RobotController` 文件夹下的 `RobotController.py` 文件中的 `kUseRos1Transfer = True` 行，该行用于指定消息传输的 ROS 版本。当值为 `True` 时，使用 ROS1 进行消息发送；当值为 `False` 时，则使用 ROS2。

people_tracking 功能包的结构如下所示：

```
people_tracking
├── model
│   ├── export_engine.sh
│   └── yolov8n_amd.engine
```

```

|   ├── yolov8n_arm.engine
|   ├── yolov8n.onnx
|   └── yolov8n.pt
└── src
    ├── GStreamerWrapper
    │   └── GStreamerWrapper.py
    ├── hub_sdk
    ├── RobotController
    │   ├── FpsCounter
    │   │   └── FpsCounter.py
    │   ├── RobotController.py
    │   ├── ROSTransfer
    │   │   ├── ROS1Transfer.py
    │   │   ├── ROS2Transfer.py
    │   │   └── TransferConstants.py
    │   └── YoloWrapper
    │       ├── CocoTypeId.py
    │       └── YoloWrapper.py
    ├── run_tracker.py
    ├── test
    │   ├── pull.py
    │   ├── pull.sh
    │   └── yolov8.py
    └── ultralytics

```

1. ***run_tracker.py*** 为人体跟随功能主程序。
2. ***GStreamerWrapper*** 是基于 DeepStream 的 GStreamer 硬件解码器，用于获取 RTSP 视频流。
3. ***RobotController.py*** 的主要运行逻辑体现在其 ***Run()*** 函数中，用于对从视频流中获取的图像进行人体识别，而后将识别结果用于发送运动指令和图像画面交互控件绘制。
4. ***ultralytics*** 下是开源的 Yolov8 程序包，用于对从视频流中获取的图像帧进行推理并进行跟踪，是 ***RobotController*** 下 ***YoloWrapper*** 的运行依赖。***ultralytics/cfg*** 文件夹是 Yolov8 各种配置文件的存放地址，各配置文件已有较完善的注释。
5. ***sdk_hub*** 下是开源的 ***sdk_hub*** 程序包，是 Yolov8 程序包的运行依赖。

7 基于激光雷达的建图定位导航避障(ROS1)

【注意】本节说明适用于 ROS1，使用前请参考“3 ROS 版本查看与切换”对 ROS 版本进行确认。

本案例使用激光雷达、感知主机实现了机器狗的建图（室内、室外场景）&定位导航功能。

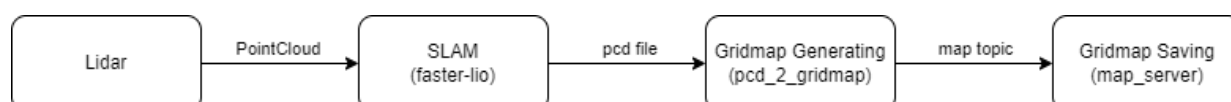
机器狗能实现在线 3D 建图和实时定位，在已有地图进行纯定位时，通过融合 IMU 实现了高速旋转、摔倒不丢失定位的性能，并使用 *move_base* 功能包实现了基于地图的导航。

【注意】开始建图前，请查看~/Desktop/version_log.txt，若版本为 v3.1.04 或更新版本，请参考 7.1 进行建图；若版本低于 v3.1.04，请参考 7.2 进行建图。

7.1 建图（适用于 v3.1.04 及以后）

7.1.1 功能介绍

本案例使用高翔博士团队在 github 上开源的 [SLAM 建图方案](#)。建图的主要操作流程和数据流程图如下所示：



7.1.2 程序结构

该建图包位于~/lite_cog/slam/src 路径下，包含 *faster-lio*、*pcd_2_gridmap*、*map_server* 三个功能包。其中 *faster-lio* 功能包负责构建三维点云地图（.pcd），*pcd_2_gridmap* 功能包负责将三维点云地图（.pcd）转为栅格地图（.pgm）并发布，*map_server* 功能包负责保存栅格地图（.pgm）。

7.1.3 使用方法

【注意】开始建图前，请检查/home/ysc/lite_cog/system/map 文件夹中是否存在以前建立的地图，如有，可移动到其他文件夹，以避免覆盖。

【注意】由于建图需要占用较多的计算资源，所以请先在 APP 上关闭所有感知功能。

1. 打开终端输入以下启动对应的激光雷达驱动：

```
1 | cd /home/ysc/lite_cog/system/scripts/lidar
2 | ./start_lslidar.sh      #如果是 Leishen 雷达，请运行该脚本
3 | ./start_livox.sh       #如果是 Livox 雷达，请运行该脚本
4 | ./start_rslidar.sh     #如果是 Robosense 雷达，请运行该脚本
```

若雷达节点启动失败，可使用以下命令检查雷达与感知主机是否建立通信连接：

```
1 | ping 192.168.1.201
```

【注意】建图过程中应保持该终端开启。

```
/home/ysc/lite_cog/drivers/leishen_ws/src/lslidar_driver/launch/lslidar_c16.launch http://localhost:11311
* /lslidar_driver_node/publish_scan: True
* /lslidar_driver_node/read_once: False
* /lslidar_driver_node/scan_num: 10
* /lslidar_driver_node/use_gps_ts: False
* /roscpp: noetic
* /rosversion: 1.16.0

NODES
/
  lslidar_driver_node (lslidar_driver/lslidar_driver_node)

ROS_MASTER_URI=http://localhost:11311

process[lslidar_driver_node-1]: started with pid [4218]
[ INFO] [1718677728.202557672]: lslidar type: c16
[ INFO] [1718677728.319704648]: filter_voxel: 1 filter_leaf_size_m: 0.05 filter_cord: 1 filter_front_m: 0.2 filter_back_m: 0.6 filter_left_m: 0.4 filter_right_m: 0.4 filter_statistical_outlier: 0 filter_statistical_mean_k: 3 filter_statistical_std: 1 filter_radius_outlier: 1 filter_radius_r: 0.05 filter_radius_count: 2
[ INFO] [1718677728.334715944]: Only accepting packets from IP address: 192.168.1.201
[ INFO] [1718677728.334944936]: Opening UDP socket: port 2368
[ INFO] [1718677728.344749128]: Only accepting packets from IP address: 192.168.1.201
[ INFO] [1718677728.345003624]: Opening UDP socket: port 2369
[ INFO] [1718677728.357309064]: return mode: 1
```

2. 启动点云建图程序：

- a) 启动建图程序的脚本 **start_slam.sh** 位于/home/ysc/lite_cog/system/scripts/slam 路径下，脚本内容如下：

```
1 | #!/bin/sh
2 |
3 | # 开启建图程序
4 | gnome-terminal -x bash -c "source /home/ysc/lite_cog/slam/devel/setup.bash;
5 |   roslaunch faster_lio mapping_c16.launch; read -p 'Press any key to exit...'"
6 |
7 | # 开启生成 grid_map 终端
```

```

8 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/gridmap.sh;
9 | read -p 'Press any key to exit...'"
10 |
11 | # 开启保存地图终端
12 | gnome-terminal -x bash -c "bash
13 | /home/ysc/lite_cog/system/scripts/slam/save_map.sh; read -p 'Press any key to
   | exit...'"

```

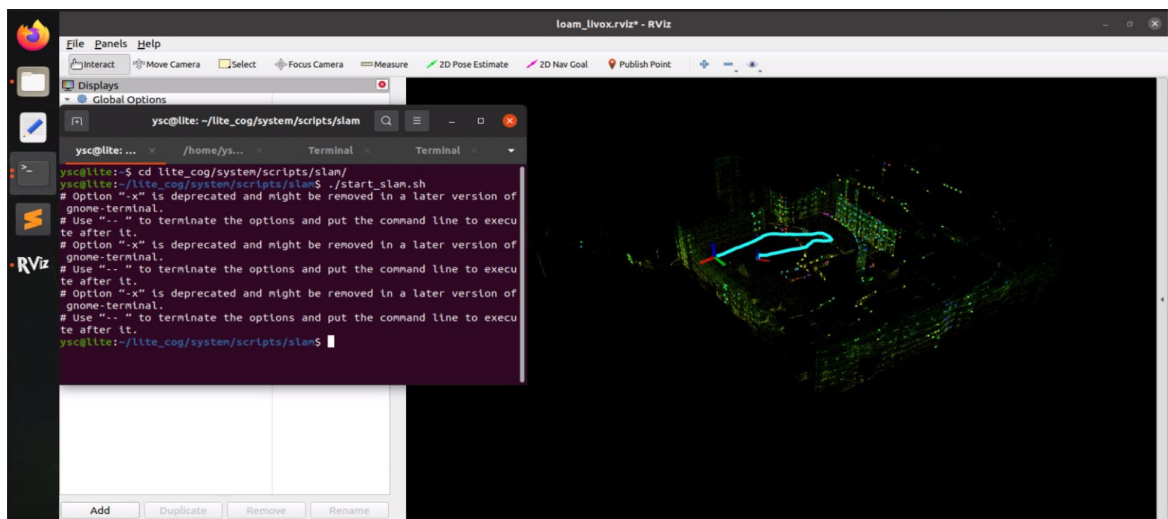
b) 用户在使用 NoMachine 登录感知主机桌面，并遥控机器狗起立后，可在感知主机中打开一个终端输入以下命令，以使用该脚本启动建图程序：

```

1 | cd /home/ysc/lite_cog/system/scripts/slam
2 | ./start_slam.sh

```

c) 执行完上述命令之后会启动可视化工具 Rviz，并在运行建图启动脚本的终端中生成 3 个终端标签页，分别用来运行 faster-lio，生成栅格地图和保存栅格地图：



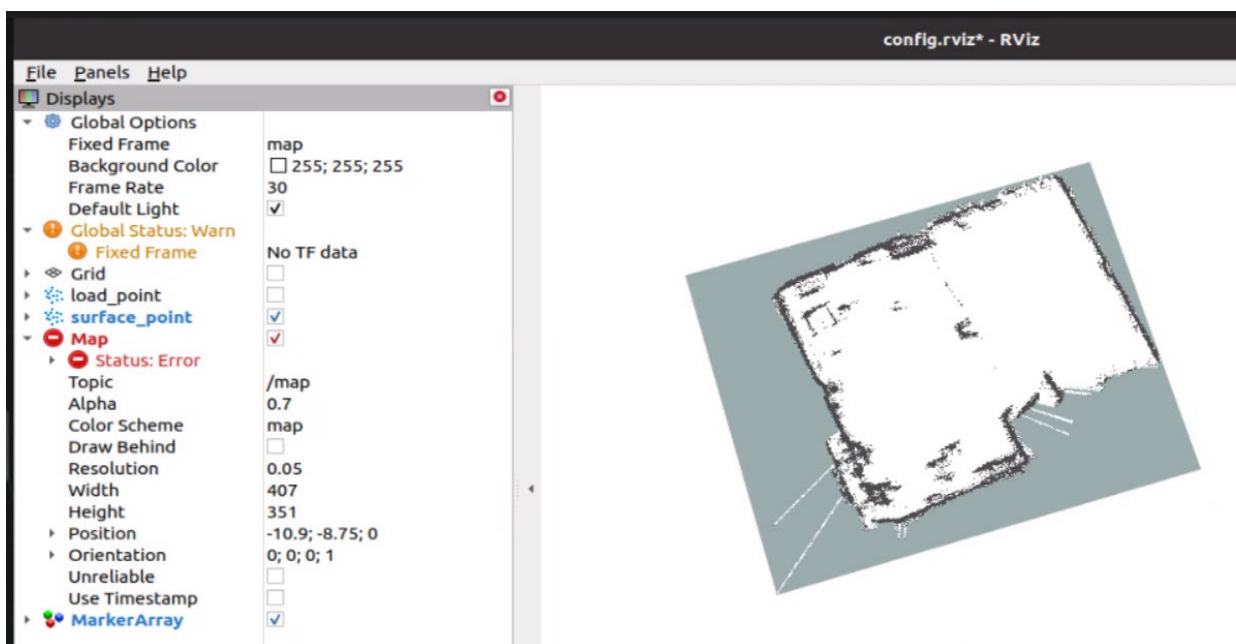
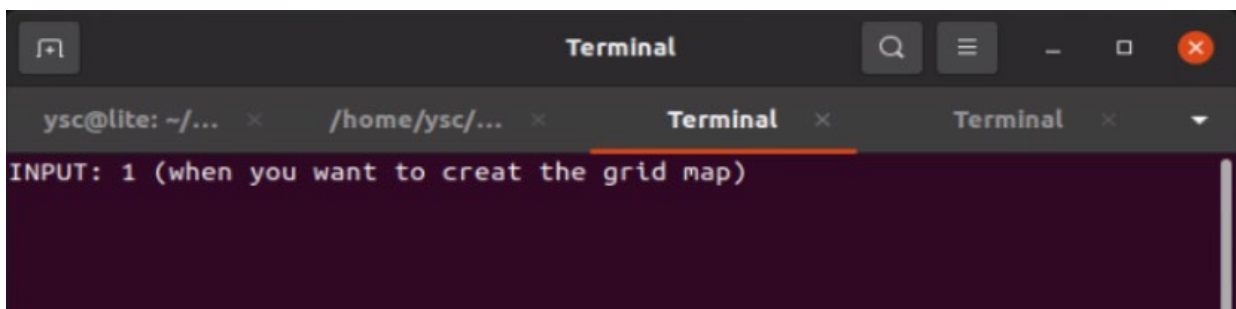
3. 遥控机器狗环绕需要构建地图的区域行走，为了建图效果，在遥控转向时，请尽量保持转速缓慢。另外，由于激光雷达存在盲区，请让机器狗与墙面保持至少 0.5 米距离。
4. 在机器狗走完需要建图的区域后，在 Rviz 中查看建立的点云地图是否与真实环境符合。
5. 点云地图扫描完成后，找到 faster-lio 程序对应的标签页，按下 ctrl+c 以停止建图，程序将自动保存三维点云文件（. pcd）到~/lite_cog/system/map 目录下，并显示统计得到的平均处理耗时（耗时仅供参考，具体数值可能有所浮动），再次按下回车键可关闭标签页。

```

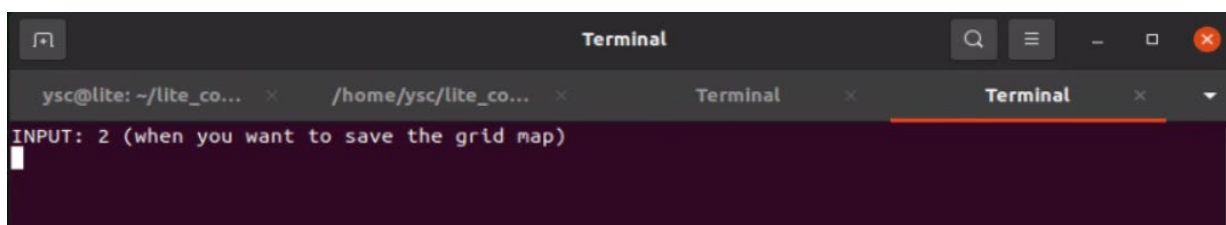
/home/ysc/lite_cog/slam/src/faster-lio/launch/mapping_c16.launch http://localhost:11311
ysc@lite: ~/lite_cog/system/sc...  /home/ysc/lite_cog/slam/src/f...  Terminal  Terminal
I0618 10:42:59.238492 13321 laser_mapping.cc:331] [ mapping ]: In num: 727 downsamp 331 Map grid num: 3240 effect num : 321
I0618 10:42:59.309013 13321 laser_mapping.cc:331] [ mapping ]: In num: 758 downsamp 334 Map grid num: 3240 effect num : 328
^C[rviz-2] killing on exit
[laserMapping-1] killing on exit
[ WARN ] [1718678579.411170120]: catch sig 2
I0618 10:42:59.437414 13321 laser_mapping.cc:331] [ mapping ]: In num: 702 downsamp 318 Map grid num: 3240 effect num : 308
I0618 10:42:59.437973 13321 run_mapping_online.cc:43] finishing mapping
I0618 10:42:59.438230 13321 laser_mapping.cc:865] current scan saved to /home/ysc/lite_cog/system/map/lite3.pcd
I0618 10:42:59.547519 13321 laser_mapping.cc:869] finish done
I0618 10:42:59.547705 13321 utils.h:52] >>> ===== Printing run time =====
I0618 10:42:59.547789 13321 utils.h:54] > [ IVox Add Points ] average time usage: 0.0595673 ms , called times: 556
I0618 10:42:59.547889 13321 utils.h:54] > [ Incremental Mapping ] average time usage: 0.260003 ms , called times: 556
I0618 10:42:59.547972 13321 utils.h:54] > [ ObsModel (IEKF Build Jacobian) ] average time usage: 0.389098 ms , called times: 2194
I0618 10:42:59.548059 13321 utils.h:54] > [ ObsModel (Lidar Match) ] average time usage: 2.21896 ms , called times: 2194
I0618 10:42:59.548146 13321 utils.h:54] > [ Downsample PointCloud ] average time usage: 0.228894 ms , called times: 556
I0618 10:42:59.548224 13321 utils.h:54] > [ IEKF Solve and Update ] average time usage: 12.7271 ms , called times: 556
I0618 10:42:59.549230 13321 utils.h:54] > [ Preprocess (Standard) ] average time usage: 0.391041 ms , called times: 560
I0618 10:42:59.551244 13321 utils.h:54] > [ Undistort Pcl ] average time usage: 1.97235 ms , called times: 553
I0618 10:42:59.551353 13321 utils.h:59] >>> ===== Printing run time end =====
I0618 10:42:59.551426 13321 run_mapping_online.cc:47] save trajectory to: ./Log/traj.txt
I0618 10:42:59.586180 13321 laser_mapping.h:34] laser mapping deconstruct
shutting down processing monitor...
... shutting down processing monitor complete
done
Press enter to exit...

```

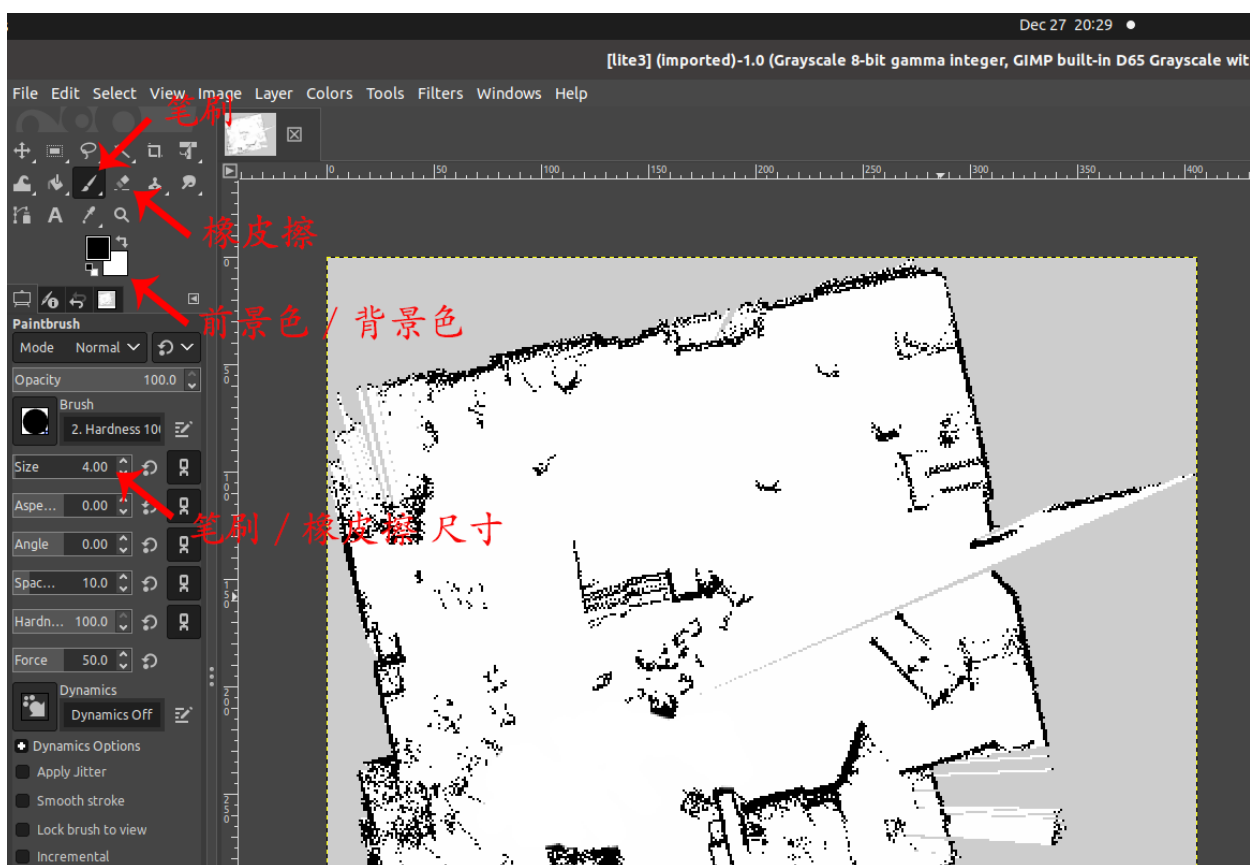
6. 三维点云文件 (.pcd) 保存成功后, 找到如下图所示的标签页, 输入 1 并按回车键后稍等一会, 此时会调用 pcd_2_gridmap 功能包将点云地图转化为栅格地图, 当弹出 RViz 窗口显示栅格地图时可进行下一步。



7. 在点云地图转化为栅格地图之后，找到如下图所示的标签页，输入数字 2 并按回车键后稍等一会，此时会调用 [map_server](#) 功能包将栅格地图保存到/home/ysc/lite_cog/system 目录下的 map 文件夹中（包括.yaml 文件、.pgm 文件）。



8. 若栅格地图不完全符合实际环境或需要人为划定可通行区域，可打开一个终端输入 `gimp` 以打开修图软件，并将/home/ysc/lite_cog/system/map 目录下的.pgm 文件拖入修图软件中进行修图。



- a) 若 GIMP Image Editor 中的工具栏没有自动打开，可在上方菜单栏中选择 Windows - New Toolbox 打开工具栏。

- b) 工具栏内含前景色与背景色工具，前景色设置铅笔工具的颜色，背景色设置橡皮擦工具的颜色。栅格地图中规定黑色为障碍物，即不可通行区域，白色为可通行区域，灰色为未知区域。您可以使用铅笔工具或橡皮擦工具添加或擦除黑色区域。
- c) 修改完成后在上方菜单栏选择 File - Overwrite usr_map.pgm 对原文件进行覆盖（请勿选择 File - Save 保存），关闭该软件时会再次出现保存提示，此时无需再保存。

9. 完成所有操作后请用 `ctrl+c` 关闭所有终端，以免影响后续进程。

10. 建好的地图文件将默认存在 `/home/ysc/lite_cog/system` 路径下的 `map` 文件中。如改变地图路径或名称，需要进行下述步骤，以便后续定位导航程序能够正确调用：

- a) 在对应的.yaml 文件中更新地图路径（如与.pgm 文件在同一路径下，则只需填写.pgm 文件名称），并修改.yaml 文件名使其与对应.pgm 文件名字保持一致：

```
1 image:/home/ysc/lite_cog/system/map.pgm           // path of map file (.pgm)
2 resolution:0.050000
3 ...
```

- b) 在 `~/lite_cog/nav/src/hdl_localization/launch` 路径下的 `local_rslidar_imu.launch` 文件中配置所需地图名称和路径：

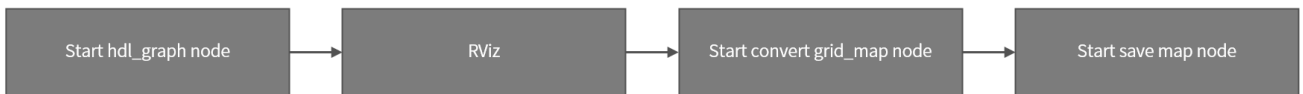
```
1 <arg name="map_name" default="lite3" />           //Define Map File Name
2 ...
3 <node name="MapServer" pkg="map_server" type="map_server"
  args="/home/ysc/lite_cog/system/map/${arg map_name}.yaml"/>
4 ...
5
6 <param name="globalmap_pcd" value="/home/ysc/lite_cog/system/map/${arg
7 map_name}.pcd" />
8 ...
```

7.2 建图（适用于 v3.1.04 以前）

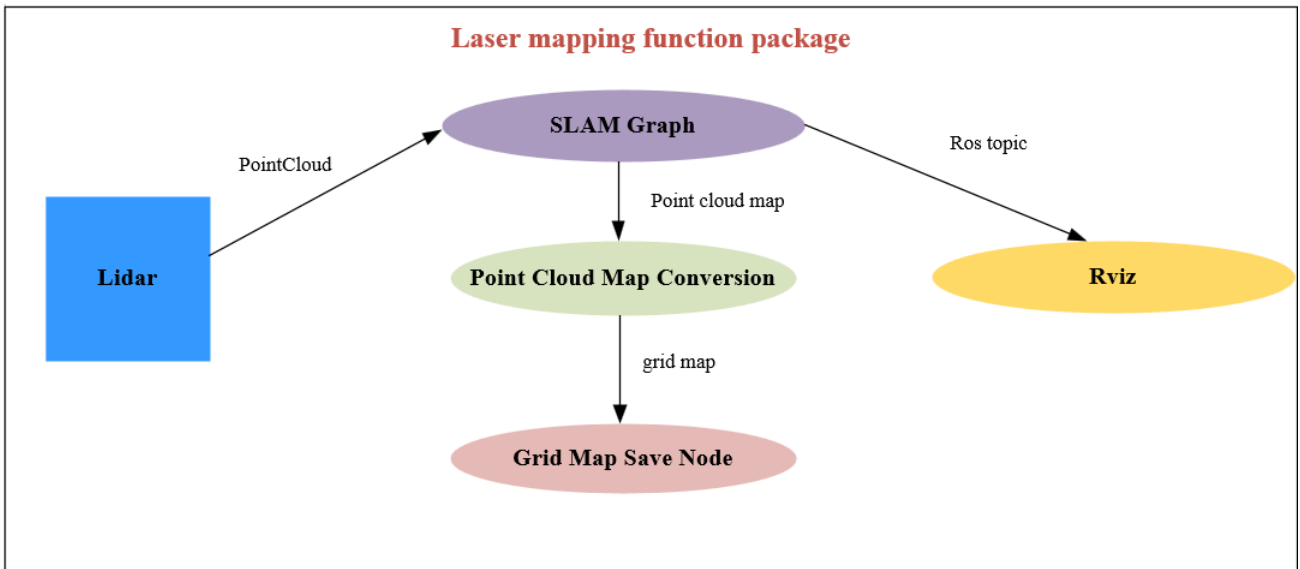
7.2.1 功能介绍

本案例使用日本丰桥科技大学的 Kenji Koide 在 github 上开源的[六自由度三维激光 SLAM 算法的室内建图方案](#)。

建图的主要操作流程如下所示：



相应的数据流图如下所示：



7.2.2 程序结构

```

/home/ysc/lite_cog/slam
├─ build
├─ devel
├─ src
│   ├── CMakeLists.txt
│   ├── fast_gicp
│   ├── hdl_graph_slam
│   ├── map_server
│   └─ ndt_omp
└─ version
  
```

hdl_graph_slam 功能包负责构建地图。

7.2.3 使用方法

【注意】开始建图前，请检查/home/ysc/lite_cog/system/map 文件夹中是否存在以前建立的地图，如有，可移动到其他文件夹，以避免覆盖。

【注意】由于建图需要占用较多的计算资源，所以请先在 APP 上关闭所有感知功能。

1. 打开终端输入以下启动对应的激光雷达驱动：

```
1 | cd /home/ysc/lite_cog/system/scripts/lidar
2 | ./start_lsldar.sh      #如果是 Leishen 雷达，请运行该脚本
3 | ./start_livox.sh      #如果是 Livox 雷达，请运行该脚本
4 | ./start_rslidar.sh    #如果是 Robosense 雷达，请运行该脚本
```

若雷达节点启动失败，可使用以下命令检查雷达与感知主机是否建立通信连接：

```
1 | ping 192.168.1.201
```

2. 启动点云建图程序：

a) 启动建图程序的脚本 **start_slam.sh** 位于/home/ysc/lite_cog/system/scripts/slam

路径下，脚本内容如下：

```
1 | #!/bin/sh
2 |
3 | # 开启 rviz
4 | gnome-terminal -x bash -c "cd /home/ysc/lite_cog/slam; source devel/setup.bash;
5 | roslaunch hdl_graph_slam mapping_rslidar_indoor.launch;"
6 |
7 | #开启 rviz
8 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/rviz.sh"
9 |
10 | #开启生成 grid_map 终端
11 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/gridmap.sh"
12 |
13 | # 开启保存地图终端
14 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/save_map.sh"
```

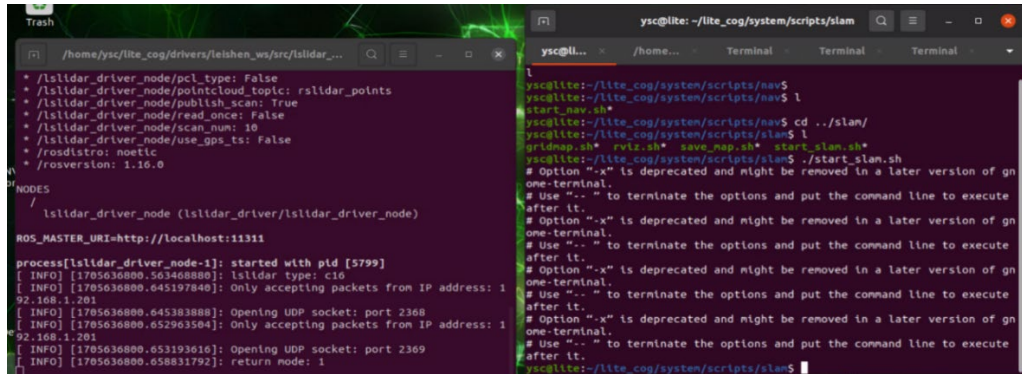
b) 用户在使用 NoMachine 登录感知主机桌面，并遥控机器狗起立后，可在感知主机中

打开一个终端输入以下命令，以使用该脚本启动建图程序：

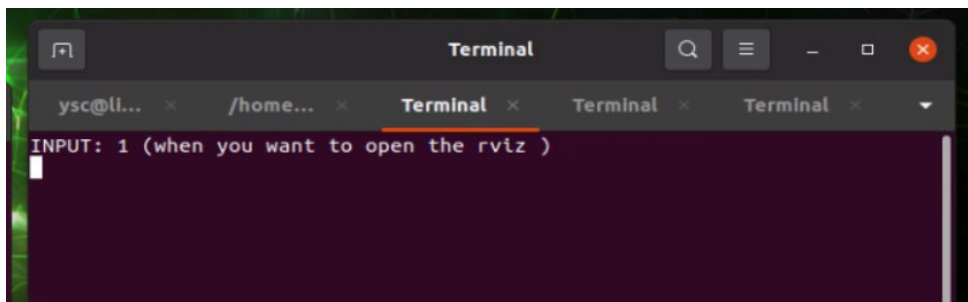
```
1 | cd /home/ysc/lite_cog/system/scripts/slam
```

```
2 | ./start_slam.sh
```

- c) 执行完上述命令之后会生成五个终端窗口，分别为运行建图脚本的终端，建图程序运行的终端，rviz 开启终端，生成栅格地图的终端，保存地图的终端（下图中左侧为雷达驱动程序窗口，右侧为建图程序脚本窗口）：

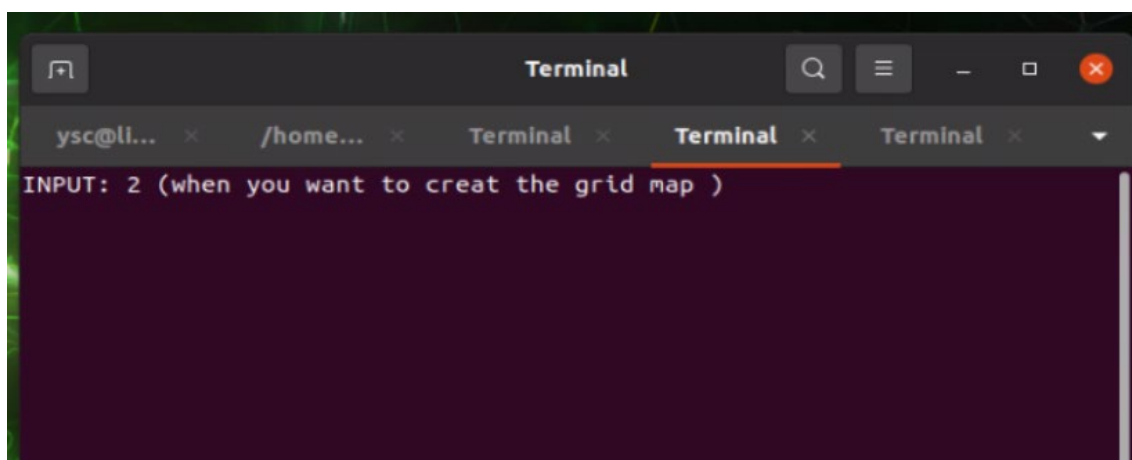


3. 若要在建图过程中实时看到建图效果，找到 RViz 可视化节点对应的终端（如下图所示），输入数字 1 并按回车键，此时会打开 RViz 可视化界面（打开此界面会降低建图性能，若后面对建图效果不满意，请尝试不打开可视化，并在建图时关闭 NoMachine 远程界面，减少算力资源占用）。

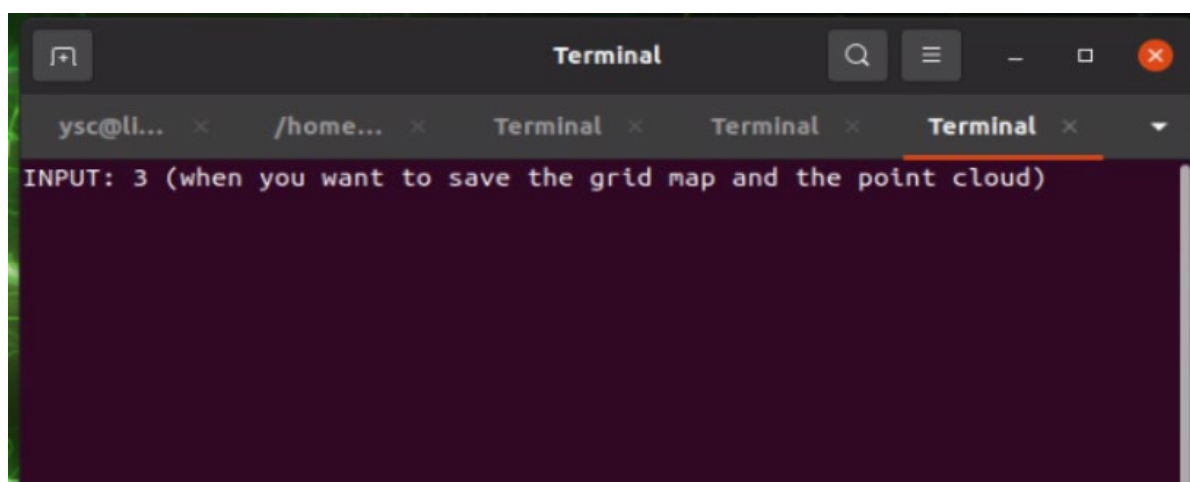


4. 遥控机器狗环绕需要构建地图的区域行走，为了建图效果，在遥控转向时，请尽量保持转速缓慢。另外，由于激光雷达存在盲区，请让机器狗与墙面保持至少 0.5 米距离。
5. 在机器狗走完需要建图的区域后，查看建立的点云地图是否与真实环境符合（若之前未打开 RViz 可视化界面，则在此时打开），若地图较大或存在回环（例如绕了房子一圈），请查看回环处的地图是否重合，若不重合，可以再绕一圈，使程序进一步完成回环检测。

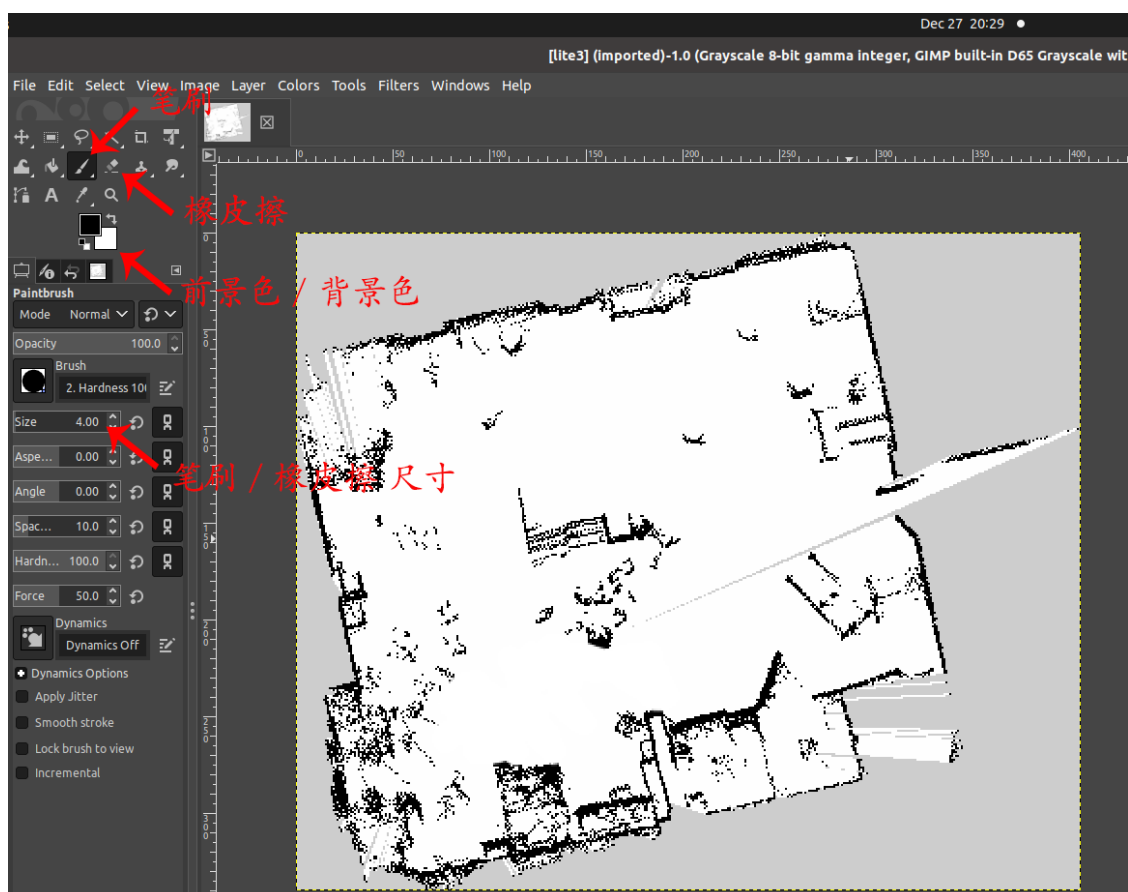
6. 点云地图扫描完成后，找到 grid map 节点对应的终端（如下图所示）输入 2 并按回车键后稍等一会，此时会调用 [octomap](#) 功能包将点云地图转化为栅格地图。



7. 在点云地图转化为栅格地图之后，找到 save map 对应的节点终端（如下图所示）输入数字 3 并按回车键后稍等一会，此时会调用 [map_server](#) 功能包保存栅格地图，相关文件将保存到/home/ysc/lite_cog/system 目录下的 map 文件夹中（包括.yaml 文件、.pgm 文件以及.pcd 文件）。



8. 若栅格地图不完全符合实际环境或需要人为划定可通行区域，可打开一个终端输入 `gimp` 以打开修图软件，并将/home/ysc/lite_cog/system/map 目录下的.pgm 文件拖入修图软件中进行修图。



a) 若 GIMP Image Editor 中的工具栏没有自动打开，可在上方菜单栏中选择 Windows - New Toolbox 打开工具栏。

b) 工具栏内含前景色与背景色工具，前景色设置铅笔工具的颜色，背景色设置橡皮擦工具的颜色。栅格地图中规定黑色为障碍物，即不可通行区域，白色为可通行区域，灰色为未知区域。您可以使用铅笔工具或橡皮擦工具添加或擦除黑色区域。

c) 修改完成后在上方菜单栏选择 File - Overwrite usr_map.pgm 对原文件进行覆盖（请勿选择 File - Save 保存），关闭该软件时会再次出现保存提示，此时无需再保存。

9. 完成所有操作后请用 `ctrl+c` 关闭所有终端，以免影响后续进程。

10. 建好的地图文件将默认存在 `/home/ysc/lite_cog/system` 路径下的 `map` 文件中。如改变地图路径或名称，需要进行下述步骤，以便后续定位导航程序能够正确调用：

- a) 在对应的.yaml 文件中更新地图路径（如与.pgm 文件在同一路径下，则只需填写.pgm 文件名称），并修改.yaml 文件名使其与对应.pgm 文件名字保持一致：

```
1 | image:/home/ysc/lite_cog/system/map.pgm           // path of map file (.pgm)
2 | resolution:0.050000
3 | ...
```

- b) 在~/lite_cog/nav/src/hdl_localization/launch 路径下的 local_rslidar_imu.launch 文件中配置所需地图名称和路径：

```
1 | <arg name="map_name" default="lite3" />           //Define Map File Name
2 | ...
3 | <node name="MapServer" pkg="map_server" type="map_server"
   | args="/home/ysc/lite_cog/system/map/$(arg map_name).yaml"/>
4 | ...
5 |
6 | <param name="globalmap_pcd" value="/home/ysc/lite_cog/system/map/$(arg
   | map_name).pcd" />
7 | ...
```

7.3 定位导航

本案例基于激光雷达和 IMU 传感器实现机器狗定位导航功能，在本案例中使用的定位算法为 [hdl_localization 定位算法](#)。

【注意】定位导航过程中需保持激光雷达驱动开启（参考 7.1.2）。

7.3.1 单目标点定位导航使用方法

1. 打开一个终端，输入以下指令以启动对应的雷达驱动（如果在建图中已经启动雷达驱动则无需再次启动）。

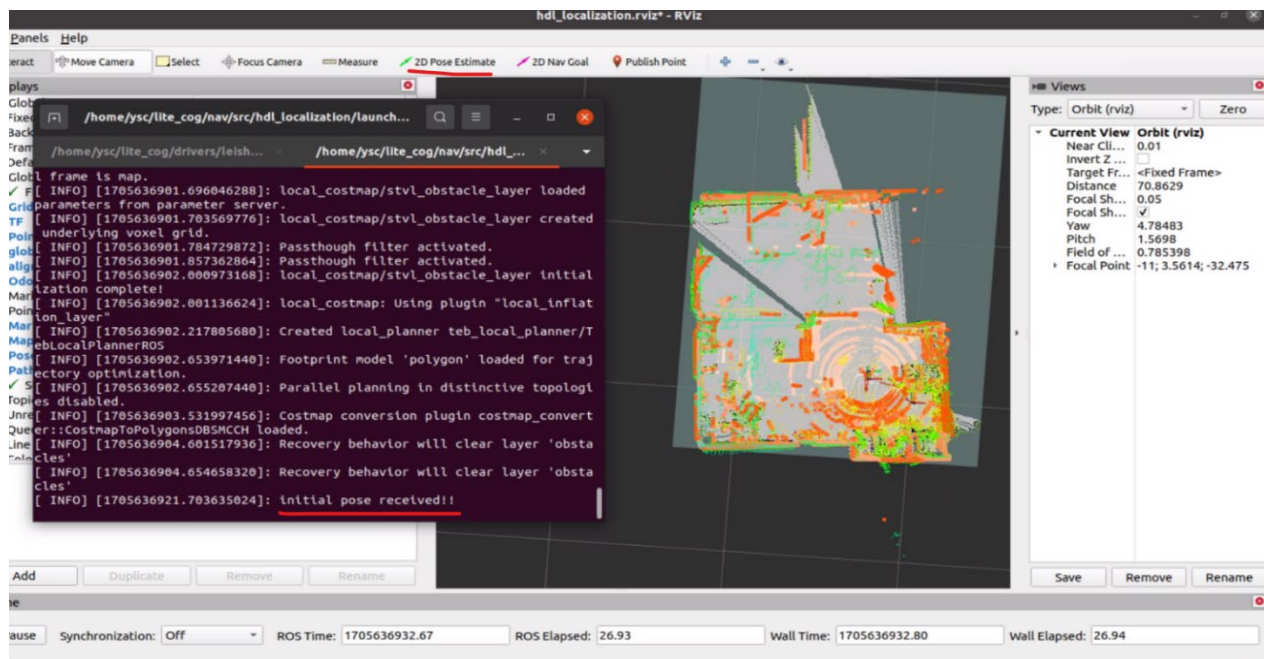
```
1 | cd /home/ysc/lite_cog/system/scripts/lidar
2 | ./start_lsldar.sh    #如果是 Leishen 雷达，请运行该脚本
3 | ./start_livox.sh     #如果是 Livox 雷达，请运行该脚本
4 | ./start_rslidar.sh   #如果是 Robosense 雷达，请运行该脚本
```

2. 打开一个终端，输入以下指令以启动定位导航节点。

```
1 | cd /home/ysc/lite_cog/system/scripts/nav
```

2 | `./start_nav.sh`

3. 开启导航后，需要对机器狗定位进行初始化：



- 点击顶部工具栏中的“2D Pose Estimate”按钮；
- 确定机器狗的实际位置和朝向，将鼠标移动到地图中的对应位置，按住左键并依照实际朝向拖拽出箭头，然后松开鼠标左键；
- 若定位初始化成功，激光点云与栅格地图会重合，且终端打印“initial pose received!!”；
- 若激光点云与栅格地图没有重合，则说明初始位置没给对，请重新操作；
- 若地图上没有出现激光点云，且终端打印的是“globalmap has not been received!!”，请用 ctrl+c 关闭程序，重新运行尝试。
- base_link 坐标系为机器狗坐标系，x 轴（红色）表示机器狗朝向。

【RViz 使用技巧】RViz 中，用鼠标滚轮可放大缩小，用鼠标左键可旋转地图，按住 shift 并使用鼠标左键，可平移拖动地图。

4. 定位初始化成功后，可按照相似方法给定一个目标点：

- 点击顶部工具栏中的“2D Nav Goal”按钮；

b) 将鼠标移动到地图中的目标位置，按住左键并依照目标朝向拖拽出箭头，然后松开鼠标；

c) 若目标点指定成功，界面中会出现规划好的机器狗运动路径；若目标点指定失败，重新操作即可。

5. 在 APP 上打开机器狗的自主模式，并使机器狗起立，机器狗将按照全局规划得到的大致路径前进，并通过局部规划避开动态障碍物，最终到达目的地。

【注意】为防止机器狗身体被认定为障碍物，只有达到一定高度的物体才会被认定为障碍物。

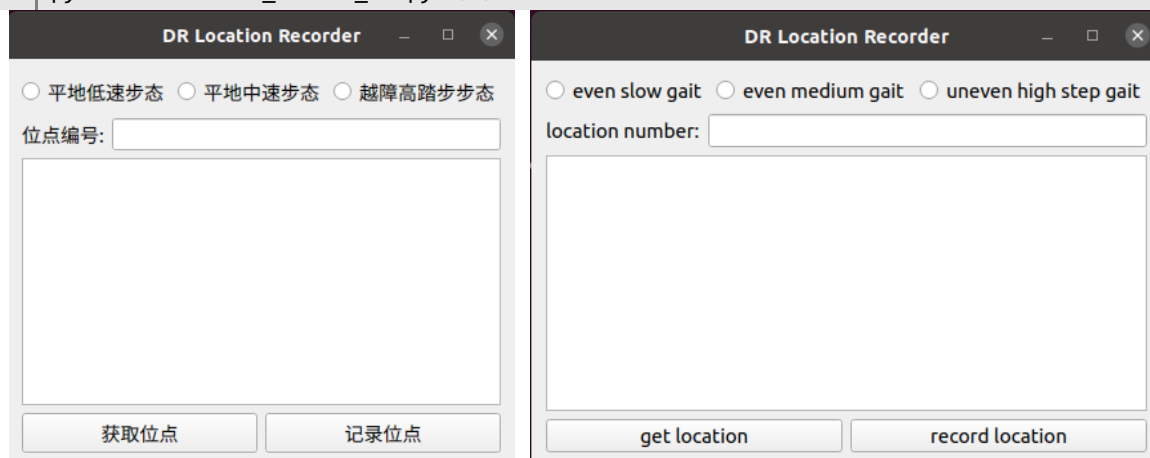
7.3.2 多目标点定位导航使用方法

本案例还提供了使机器狗按序到达一系列目标点进行循环导航的功能。

【注意】开始标点前，请检查/home/ysc/lite_cog/pipeline/src/pipeline/data 文件夹中是否存在以前记录的目标点文件，如有，可移动到其他文件夹，以避免覆盖。

1. 首先参照 7.3.1 中的第 1 步至第 3 步启动导航程序并给定初始定位，然后再打开一个终端，运行以下指令开启目标点配置程序：

```
1 cd /home/ysc/lite_cog/pipeline/src/pipeline_tracking/tools
2 python3 location_record.py
3 python3 location_record_en.py #英文版
```



2. 将机器狗手动遥控至某个目标点位置，在“位点编号” / “location number” 处输入 1，点击“获取点位” / “get location”，再点击“记录点位” / “record location”，在 `/home/ysc/lite_cog/pipeline/src/pipeline/data` 目录下会出现一个名为 **1.json** 的记录文件。然后将机器狗要遥控至下一个位置，重复上述操作，直至所有点位记录完毕后，关闭标点程序。若中途标点程序被关闭，再次打开即可（参考第 1 步）。
3. 打开一个终端，运行以下命令，并在 APP 上将机器狗的自主模式打开，机器狗将前往距离最近的记忆位点，并按照位点序号循环导航。

```
1 | cd /home/ysc/lite_cog/pipeline
2 | source devel/setup.bash
3 | cd /home/ysc/lite_cog/pipeline/src/pipeline_tracking/scripts
4 | python3 Task.py
```

4. 此后，再次使用时，只需要先参照 7.3.1 启动导航程序并给定初始定位，然后执行第 3 步，即可使机器狗按照先前记录的目标点进行循环导航。

7.4 二次开发说明

7.4.1 雷达驱动二次开发

雷达驱动位于 `~/lite_cog/driver/leishen_ws/` 和 `~/lite_cog/driver/mid360_ws/` 以及 `~/lite_cog/driver/rslidar_sdk/` 目录下。用户可自行阅读源码进行二次开发。

7.4.2 SLAM 建图二次开发

SLAM 建图程序通过 `/home/ysc/lite_cog/system/scripts/slam` 文件夹下的 `start_slam.sh` 文件进行启动，`start_slam.sh` 文件中第一行为启动建图程序，相应的 launch 文件中包含参数配置文件，其中主要包含点云预处理参数、点云配准算法参数、点云保存参数等，用户可自行调整。

7.4.3 定位导航二次开发

1. 定位二次开发：定位导航程序通过/home/ysc/lite_cog/system/scripts/nav 文件夹下的 start_nav.sh 文件进行启动，通过 start_nav.sh 文件中的内容可以找到定位启动的 launch 文件，其中主要包含点云预处理参数和点云配准参数，用户可自行调整。
2. 导航二次开发：在 /home/ysc/lite_cog/nav/src/navigation/config 目录下有 6 个.yaml 文件可以进行参数调整，分别是：
 - a) common_costmap_params.yaml: 包含局部代价地图和全局代价地图共用参数；
 - b) global_costmap_params.yaml: 包含全局代价地图参数；
 - c) local_costmap_params.yaml: 包含局部代价地图参数；
 - d) global_planner_params.yaml: 包含全局规划器参数；
 - e) teb_local_planner_params.yaml: 包含 teb local planner 局部规划器参数；
 - f) move_base_params.yaml: 包含 move_base 软件包参数。

用户可点击链接阅读学习 global_planner、teb_local_planner 和 costmap_2d 等软件包相关说明：[global_planner 文档](#)、[teb_local_planner 文档](#)、[costmap_2d 文档](#)、[move_base 文档](#)。

8 基于激光雷达的建图定位导航避障(ROS2)

【注意】本节说明适用于 ROS2，使用前请参考“3 ROS 版本查看与切换”对 ROS 版本进行确认。

本案例使用激光雷达、感知主机实现了机器狗的建图（室内、室外场景）&定位导航功能。

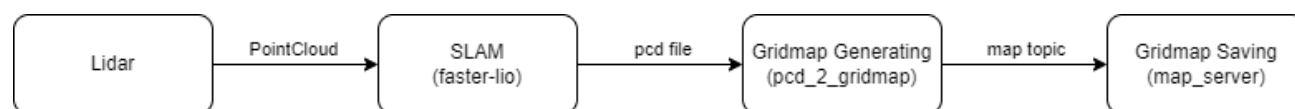
机器狗能实现在线 3D 建图和实时定位，在已有地图进行纯定位时，通过融合 IMU 实现了高速旋转、摔倒不丢失定位的性能，并使用 *bt_navigator* 功能包实现了基于地图的导航。

8.1 建图

8.1.1 功能介绍

本案例使用高翔博士团队在 github 上开源的 [SLAM 建图方案](#)。

建图的主要操作流程和数据流图如下所示：



8.1.2 程序结构

```

ysc@lite:~/lite_cog_ros2/slam/src$ tree .
.
├── faster-lio
├── map_server
└── pcd2grid
  
```

其中 *faster-lio* 功能包负责构建 pcd 地图、*pcd2grid* 功能包负责将 pcd 地图转为栅格地图并发布，*map_server* 功能包负责保存栅格地图。

8.1.3 使用方法

【注意】开始建图前，请检查/home/ysc/lite_cog_ros2/system/map 文件夹中是否存在以前建立的地图，如有，可移动到其他文件夹，以避免覆盖。

【注意】由于建图需要占用较多的计算资源，所以请先在 APP 上关闭所有感知功能。

1. 打开终端输入以下启动对应的激光雷达驱动：

```
1 | cd ~/lite_cog_ros2/system/scripts/lidar
2 | ./start_lsldar.sh    #如果是 Leishen 雷达，请运行该脚本
3 | ./start_livox.sh     #如果是 Livox 雷达，请运行该脚本
4 | ./start_rslidar.sh   #如果是 Robosense 雷达，请运行该脚本
```

若雷达节点启动失败，可使用以下命令检查雷达与感知主机是否建立通信连接：

```
1 | ping 192.168.1.201
```

2. 启动点云建图程序：

a) 启动建图程序的脚本 start_slam.sh 位于~/lite_cog_ros2/system/scripts/slam 路径

下，脚本内容如下：

```
1 | #!/bin/sh
2 |
3 | # 开启 rviz
4 | gnome-terminal -x bash -c "source /home/ysc/lite_cog_ros2/slam/install/setup.bash;
5 | ros2 launch faster_lio mapping_c16.launch.py; read -p 'Press Enter to exit...'"
6 |
7 | #开启生成 grid_map 终端
8 | gnome-terminal -x bash -c "bash
9 | /home/ysc/lite_cog_ros2/system/scripts/slam/gridmap.sh; read -p 'Press Enter to
10 | exit...'"
11 |
12 | # 开启保存地图终端
13 | gnome-terminal -x bash -c "bash
14 | /home/ysc/lite_cog_ros2/system/scripts/slam/save_map.sh; read -p 'Press any key to
15 | exit...'; read -p 'Press Enter to exit...'"
```

b) 用户在使用 NoMachine 登录感知主机桌面，并遥控机器狗起立后，可在感知主机中

打开一个终端输入以下命令，以使用该脚本启动建图程序：

```
1 | cd ~/lite_cog_ros2/system/scripts/slam
2 | ./start_slam.sh
```

c) 执行完上述命令之后会生成四个终端窗口，分别为运行建图脚本的终端，建图程序运行的终端，生成栅格地图的终端，保存地图的终端（下图中左侧为雷达驱动程序窗口，右侧为建图程序脚本窗口）：

```

ysc@lite: ~/lite_cog_ros2/system/scripts/lidar
coordinate_opt: 0
[lslidar_driver_node-2] [INFO] [1723513050.792524096] [c16.lslidar_driver_node]:
[driver][input] accepting packets from IP address: 192.168.1.201 port: 2368
[lslidar_driver_node-2] [INFO] [1723513050.792817791] [c16.lslidar_driver_node]:
[driver][input] accepting packets from IP address: 192.168.1.201 port: 2369
[lslidar_driver_node-2] [INFO] [1723513050.793171327] [c16.lslidar_driver_node]:
[driver][socket] opening UDP socket: port 2369
[lslidar_driver_node-2] [INFO] [1723513050.793727192] [c16.lslidar_driver_node]:
lidar: c16
[lslidar_driver_node-2] [INFO] [1723513051.068734395] [c16.lslidar_driver_node]:
total working time: 21 hours:12 minutes
[lslidar_driver_node-2] [INFO] [1723513051.070624952] [c16.lslidar_driver_node]:
less than -40 degrees working time: 0 hours:8 minutes
[lslidar_driver_node-2] [INFO] [1723513051.07113623] [c16.lslidar_driver_node]:
-40 degrees - -10 degrees working time: 0 hours:0 minutes
[lslidar_driver_node-2] [INFO] [1723513051.071787830] [c16.lslidar_driver_node]:
-10 degrees - 30 degrees working time: 1 hours:43 minutes
[lslidar_driver_node-2] [INFO] [1723513051.072010134] [c16.lslidar_driver_node]:
30 degrees - 70 degrees working time: 19 hours:29 minutes
[lslidar_driver_node-2] [INFO] [1723513051.072102774] [c16.lslidar_driver_node]:
70 degrees - 100 degrees working time: 0 hours:0 minutes
[lslidar_driver_node-2] [INFO] [1723513051.069589466] [c16.lslidar_driver_node]:
return mode: 1

ysc@lite: ~/lite_cog_ros2/system/scripts/slam
ysc@lite: ~/lite_cog_ros2/system/scripts/slam$ ./start_slam.sh
# Option "-x" is deprecated and might be removed in a later version of gnome-ter
# Use "--" to terminate the options and put the command line to execute after t
# Option "-x" is deprecated and might be removed in a later version of gnome-ter
# Use "--" to terminate the options and put the command line to execute after t
# Option "-x" is deprecated and might be removed in a later version of gnome-ter
# Use "--" to terminate the options and put the command line to execute after t
ysc@lite: ~/lite_cog_ros2/system/scripts/slam$

```

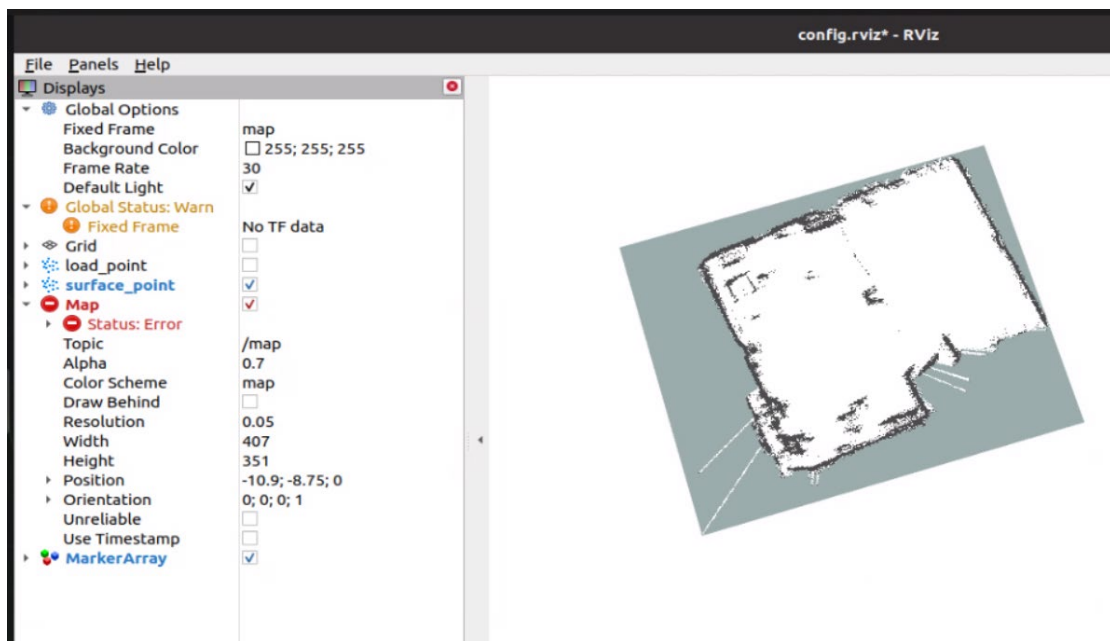
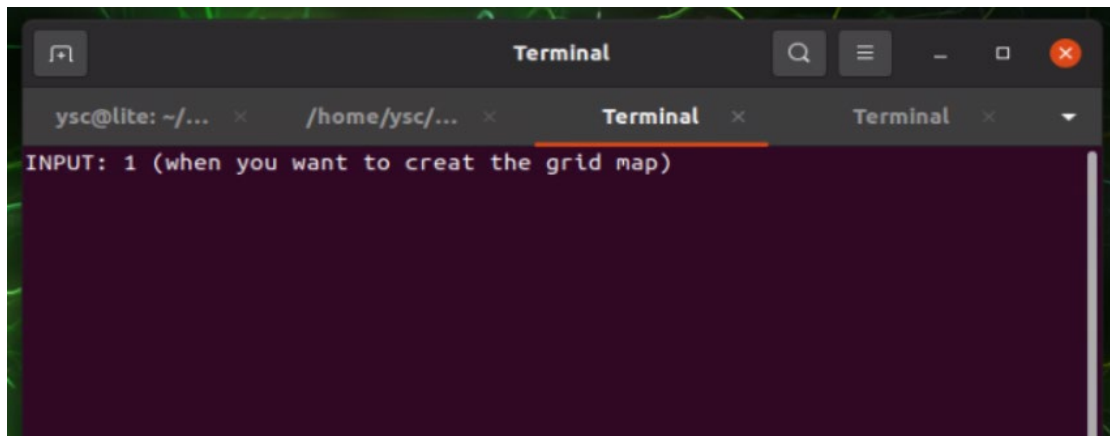
3. 遥控机器狗环绕需要构建地图的区域行走，为了建图效果，在遥控转向时，请尽量保持转速缓慢。另外，由于激光雷达存在盲区，请让机器狗与墙面保持至少 0.5 米距离。
4. 在机器狗走完需要建图的区域后，查看建立的点云地图是否与真实环境符合。
5. 点云地图扫描完成后，找到 faster-lio 程序对应的中断输入 ctrl+c 停止建图，程序将自动保存 pcd 文件到~/lite_cog_ros2/system/map 目录下，并显示统计得到的平均处理耗时（耗时仅供参考，具体数值可能有所浮动），再次按下回车键可退出终端。

```

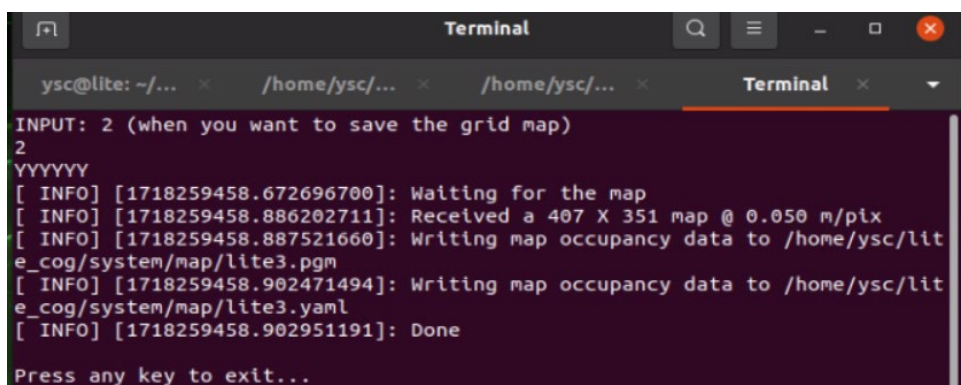
Terminal
ysc@lite: ~/lite_cog_ros2/system/scri...
[run_mapping_online-1] 10813 09:39:37.641146 4530 laser_mapping.cc:386 [ mapping ]: In num: 1986 downsamp 548 Map grid num: 1236 effect num : 547
[run_mapping_online-1] 10813 09:39:37.733299 4530 laser_mapping.cc:386 [ mapping ]: In num: 1972 downsamp 520 Map grid num: 1236 effect num : 519
[run_mapping_online-1] 10813 09:39:37.827049 4530 laser_mapping.cc:386 [ mapping ]: In num: 1982 downsamp 505 Map grid num: 1236 effect num : 504
[run_mapping_online-1] 10813 09:39:37.946548 4530 laser_mapping.cc:386 [ mapping ]: In num: 1970 downsamp 517 Map grid num: 1236 effect num : 516
[run_mapping_online-1] 10813 09:39:38.045049 4530 laser_mapping.cc:386 [ mapping ]: In num: 1976 downsamp 556 Map grid num: 1236 effect num : 555
[run_mapping_online-1] 10813 09:39:38.137085 4530 laser_mapping.cc:386 [ mapping ]: In num: 1958 downsamp 543 Map grid num: 1236 effect num : 542
[run_mapping_online-1] 10813 09:39:38.230703 4530 laser_mapping.cc:386 [ mapping ]: In num: 1977 downsamp 513 Map grid num: 1236 effect num : 512
[run_mapping_online-1] 10813 09:39:38.328855 4530 laser_mapping.cc:386 [ mapping ]: In num: 1955 downsamp 549 Map grid num: 1236 effect num : 548
[run_mapping_online-1] 10813 09:39:38.452644 4530 laser_mapping.cc:386 [ mapping ]: In num: 1952 downsamp 549 Map grid num: 1236 effect num : 549
^C[WARNING] [launch]: user interrupted with ctrl-c (SIGINT)
[rviz2-2] [INFO] [1723513178.471607456] [rclcpp]: signal_handler(signal_value=2)
[run_mapping_online-1] catch sig 2
[run_mapping_online-1] 10813 09:39:38.480989 4530 run_mapping_online.cc:47] finishing mapping
[run_mapping_online-1] 10813 09:39:38.481072 4530 laser_mapping.cc:946] current scan saved to /home/ysc/lite_cog_ros2/system/map/lite3.pcd
[run_mapping_online-1] 10813 09:39:38.850165 4530 laser_mapping.cc:950] finish done
[run_mapping_online-1] 10813 09:39:38.850272 4530 utils.h:52] >>> ===== Printing run time =====
[run_mapping_online-1] 10813 09:39:38.850311 4530 utils.h:54] > [ Ivox Add Points ] average time usage: 0.0108929 ms , called times: 1180
[run_mapping_online-1] 10813 09:39:38.850376 4530 utils.h:54] > [ Incremental Mapping ] average time usage: 0.181746 ms , called times: 1180
[run_mapping_online-1] 10813 09:39:38.850420 4530 utils.h:54] > [ ObsModel (IEKF Build Jacobian) ] average time usage: 0.237562 ms , called times: 4265
[run_mapping_online-1] 10813 09:39:38.850505 4530 utils.h:54] > [ ObsModel (Lidar Match) ] average time usage: 1.74629 ms , called times: 4265
[run_mapping_online-1] 10813 09:39:38.850587 4530 utils.h:54] > [ Downsample PointCloud ] average time usage: 0.448277 ms , called times: 1180
[run_mapping_online-1] 10813 09:39:38.850651 4530 utils.h:54] > [ IEKF Solve and Update ] average time usage: 9.20308 ms , called times: 1180
[run_mapping_online-1] 10813 09:39:38.850695 4530 utils.h:54] > [ Preprocess (Standard) ] average time usage: 0.573964 ms , called times: 1182
[run_mapping_online-1] 10813 09:39:38.850739 4530 utils.h:54] > [ Undistort Pcl ] average time usage: 2.444 ms , called times: 1181
[run_mapping_online-1] 10813 09:39:38.850780 4530 utils.h:59] >>> ===== Printing run time end =====
[run_mapping_online-1] 10813 09:39:38.850812 4530 run_mapping_online.cc:51] save trajectory to: ./Log/traj.txt
[run_mapping_online-1] 10813 09:39:38.850854 4530 laser_mapping.cc:950] finish done
[run_mapping_online-1] 10813 09:39:38.851735 4530 laser_mapping.h:35] laser mapping deconstruct
[INFO] [run_mapping_online-1]: process has finished cleanly [pid 4530]
[INFO] [rviz2-2]: process has finished cleanly [pid 4532]
Press Enter to exit...

```

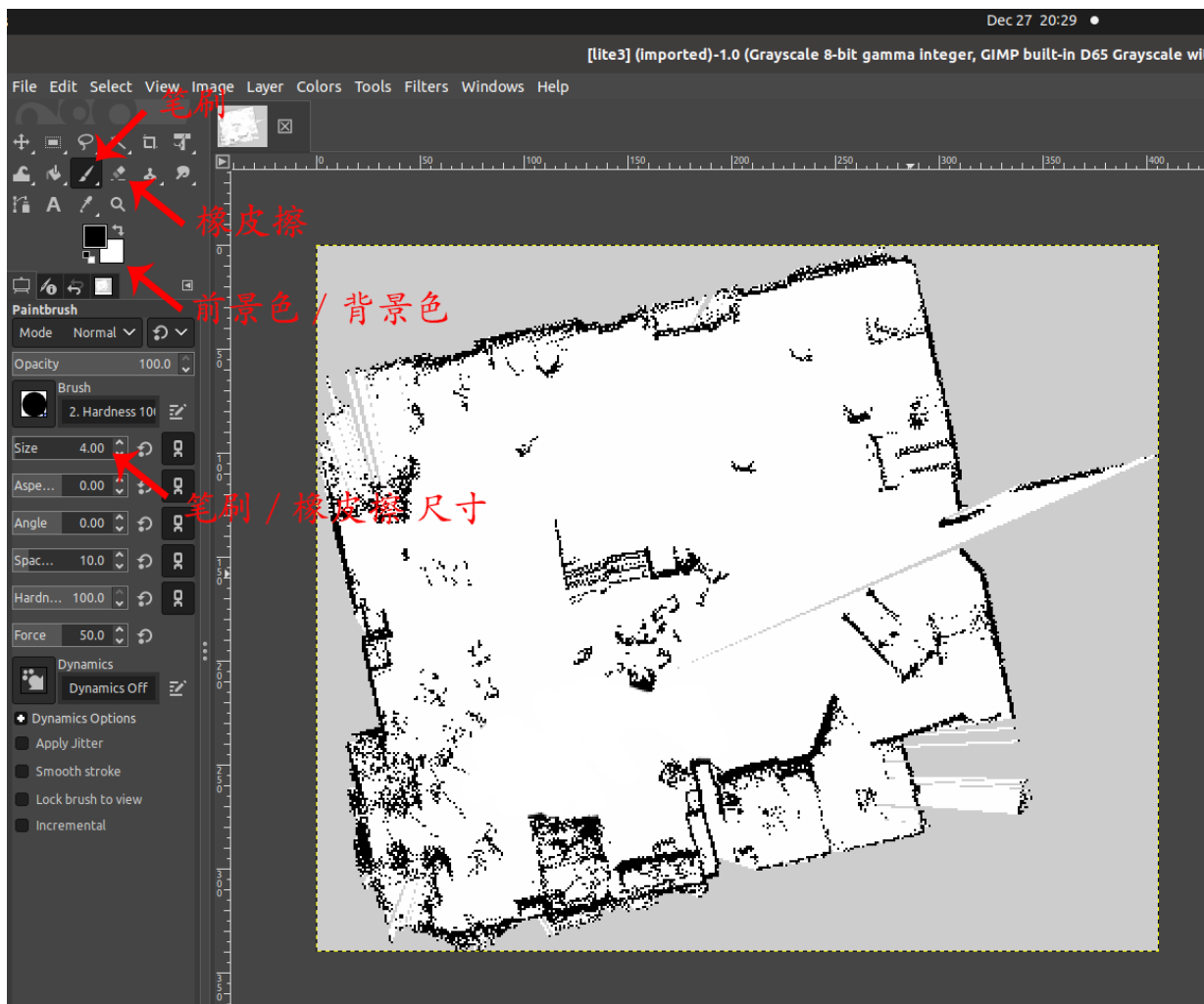
6. pcd 文件保存成功后，找到 grid map 节点对应的终端（如下图所示）输入 1 并按回车键后稍等一会，此时会调用 pcd_2_gridmap 功能包将点云地图转化为栅格地图，当弹出 RViz 窗口显示栅格地图时可进行下一步。



7. 在点云地图转化为栅格地图之后，找到 save map 对应的节点终端（如下图所示）输入数字 2 并按回车键后稍等一会，此时会调用 map_server 功能包保存栅格地图，文件将保存到/home/ysc/lite_cog_ros2/system 目录下的 map 文件夹中（包括.yaml 文件、.pgm 文件）。



8. 若栅格地图不完全符合实际环境或需要人为划定可通行区域，可打开一个终端输入 gimp 以打开修图软件，并将/home/ysc/lite_cog_ros2/system/map 目录下的.pgm 文件拖入修图软件中进行修图。



- a) 若 GIMP Image Editor 中的工具栏没有自动打开，可在上方菜单栏中选择 Windows - New Toolbox 打开工具栏。
- b) 工具栏内含前景色与背景色工具，前景色设置铅笔工具的颜色，背景色设置橡皮擦工具的颜色。栅格地图中规定黑色为障碍物，即不可通行区域，白色为可通行区域，灰色为未知区域。您可以使用铅笔工具或橡皮擦工具添加或擦除黑色区域。

- c) 修改完成后在上方菜单栏选择 File - Overwrite usr_map.pgm 对原文件进行覆盖（请勿选择 File - Save 保存），关闭该软件时会再次出现保存提示，此时无需再保存。

9. 完成所有操作后请用 `ctrl+c` 关闭所有终端，以免影响后续进程。

10. 建好的地图文件将默认存在 `/home/ysc/lite_cog_ros2/system` 路径下的 `map` 文件中。

如改变地图路径或名称，需要进行下述步骤，以便后续定位导航程序能够正确调用：

- a) 在对应的.yaml 文件中更新地图路径（如与.pgm 文件在同一路径下，则只需填写.pgm 文件名称），并修改.yaml 文件名使其与对应.pgm 文件名字保持一致：

```
1 image:/home/ysc/lite_cog_ros2/system/map.pgm           // path of map file (.pgm)
2 resolution:0.050000
3 ...
```

- b) 在 `~/lite_cog_ros2/nav/src/hdl_localization/launch` 路径下的 `lite_localization.launch.py` 文件中配置所需地图名称和路径：

```
1 declare_map_server_config_file_cmd = DeclareLaunchArgument(
2     'map_server_config_file',
3     default_value=os.path.join(
4         '/home/ysc/lite_cog_ros2/system/map/lite3.yaml'
5     )
6 )
7
8 ...
9 parameters=[
10     {"globalmap_pcd": "/home/ysc/lite_cog_ros2/system/map/lite3.pcd"},
11     ...
```

8.2 定位导航

本案例基于激光雷达和 IMU 传感器实现机器狗定位导航功能，在本案例中使用的定位算法为

[hdl_localization 定位算法](#)。

【注意】 定位导航过程中需保持激光雷达驱动开启。

8.2.1 单目标点定位导航使用方法

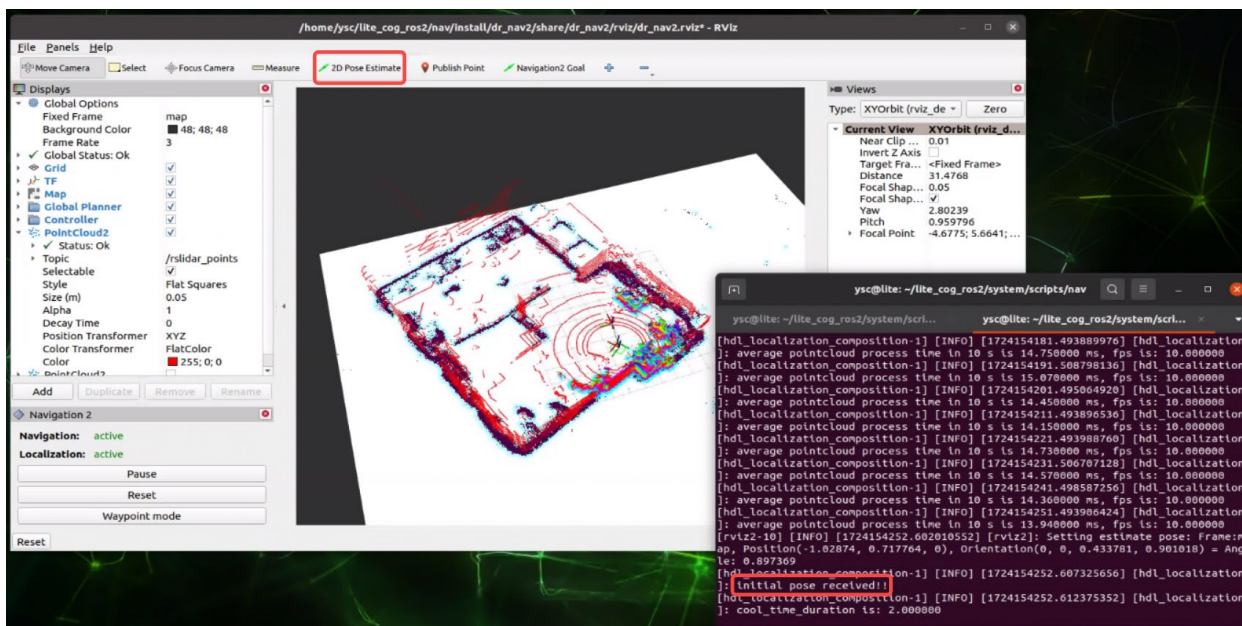
1. 打开一个终端，根据所使用雷达种类输入以下指令以启动对应的雷达驱动（如果在建图中已经启动雷达驱动则无需再次启动）。

```
1 | cd ~/lite_cog_ros2/system/scripts/lidar
2 | ./start_lsldar.sh    #如果是 Leishen 雷达，请运行该脚本
3 | ./start_livox.sh     #如果是 Livox 雷达，请运行该脚本
4 | ./start_rslidar.sh   #如果是 Robosense 雷达，请运行该脚本
```

2. 打开一个终端，输入以下指令以启动定位导航节点。

```
1 | cd /home/ysc/lite_cog_ros2/system/scripts/nav
2 | ./start_nav.sh
```

3. 开启导航后，需要对机器狗定位进行初始化：



- a) 点击顶部工具栏中的“2D Pose Estimate”按钮；
- b) 确定机器狗的实际位置和朝向，将鼠标移动到地图中的对应位置，按住左键并依照实际朝向拖拽出箭头，然后松开鼠标左键；
- c) 若定位初始化成功，激光点云与栅格地图会重合，且终端打印“initial pose received!!”；
- d) 若激光点云与栅格地图没有重合，则说明初始位置没给对，请重新操作；

e) 若地图上没有出现激光点云，且终端打印的是“globalmap has not been received!!”，请用 ctrl+c 关闭程序，重新运行尝试。

f) base_link 坐标系为机器狗坐标系，x 轴（红色）表示机器狗朝向。

【RViz 使用技巧】RViz 中，用鼠标滚轮可放大缩小，用鼠标左键可旋转地图，按住 shift 并长按鼠标左键，可平移拖动地图。

4. 定位初始化成功后，可按照相似方法给定一个目标点：

a) 点击顶部工具栏中的“Navigation2 Goal”按钮；

b) 将鼠标移动到地图中的目标位置，按住左键并依照目标朝向拖拽出箭头，然后松开鼠标；

c) 若目标点指定成功，界面中会出现规划好的机器狗运动路径；若目标点指定失败，重新操作即可。

5. 在 APP 上打开机器狗的自主模式，并使机器狗起立，机器狗将按照全局规划得到的大致路径前进，并通过局部规划避开动态障碍物，最终到达目的地。

【注意】为防止机器狗身体被认定为障碍物，只有达到一定高度的物体才会被认定为障碍物。

8.2.2 多目标点定位导航使用方法

本案例还提供了使机器狗按序到达一系列目标点进行循环导航的功能。

【注意】开始标点前，请检查/home/ysc/lite_cog_ros2/pipeline/src/pipeline/data 文件夹中是否存在以前记录的目标点文件，如有，可移动到其他文件夹，以避免覆盖。

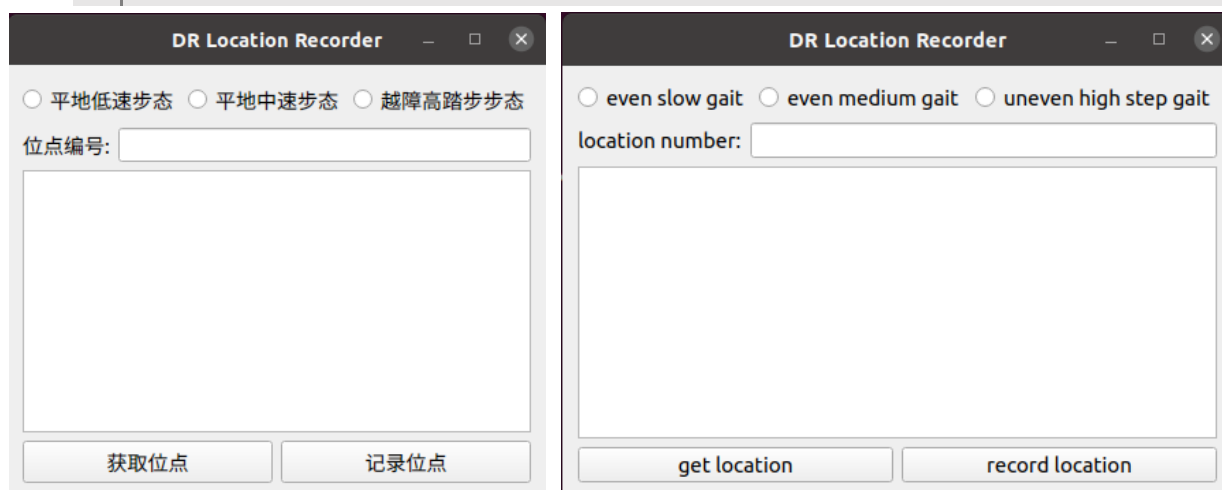
1. 首先参照 8.2.1 中的第 1 步至第 3 步启动导航程序并给定初始定位，然后再打开一个终端，运行以下指令开启目标点配置程序：

a) 如需打开中文版界面，请运行：

```
1 cd ~/lite_cog_ros2/pipeline/src/pipeline
2 python3 LocationRecorder.py
```

b) 如需打开英文版界面，请运行：

```
1 | cd ~/lite_cog_ros2/pipeline/src/pipeline
2 | python3 LocationRecorder_EN.py
```



2. 将机器狗手动遥控至某个目标点位置，在“位点编号” / “location number” 处输入 1，点击“获取点位” / “get location”，再点击“记录点位” / “record location”，在/home/ysc/lite_cog_ros2/pipeline/src /data 目录下会出现一个名为 **1.json** 的记录文件。然后将机器狗要遥控至下一个位置，重复上述操作，直至所有点位记录完毕后，关闭标点程序。若中途标点程序被关闭，再次打开即可。
3. 打开一个终端，运行以下命令，并在 APP 上将机器狗的自主模式打开，机器狗将前往距离最近的记忆位点，并按照位点序号循环导航。

```
1 | cd /home/ysc/lite_cog_ros2/pipeline/src/pipeline
2 | python3 Task.py
```

4. 此后，再次使用时，只需要执行第 1 步和第 3 步，即可使机器狗按照先前记录的目标点进行循环导航。

8.3 二次开发说明

8.3.1 雷达驱动二次开发

雷达驱动位于~/lite_cog_ros2/driver/leishen_ws/和~/lite_cog_ros2/driver/mid360_ws/以及~/lite_cog_ros2/driver/rslidar_sdk/目录下。用户可自行阅读源码进行二次开发。

8.3.2 SLAM 建图二次开发

SLAM 建图程序通过/home/ysc/lite_cog_ros2/system/scripts/slam 文件夹下的 start_slam.sh 文件进行启动，start_slam.sh 文件中第一行为启动建图程序，相应的 launch 文件中包含参数配置文件，其中主要包含点云预处理参数、点云配准算法参数、点云保存参数等，用户可自行调整。

8.3.3 定位导航二次开发

1. 定位二次开发：定位导航程序通过/home/ysc/lite_cog_ros2/system/scripts/nav 文件夹下的 start_nav.sh 文件进行启动，通过 start_nav.sh 文件中的内容可以找到定位启动的 launch 文件，其中主要包含点云预处理参数和点云配准参数，用户可自行调整。
2. 导航二次开发：在 /home/ysc/lite_cog_ros2/nav/src/dr_nav2/config 目录下的 lite_nav2.yaml 文件中可以进行参数调整，各参数的作用可参考 [navigation2 官方文档](#)。